IEC 61360-1

Edition 4.0 2017-07

INTERNATIONAL STANDARD

NORME INTERNATIONALE

colour inside

Standard data element types with associated classification scheme – Part 1: Definitions – Principles and methods

Types normalisés d'éléments de données avec plan de classification – Partie 1: Définitions – Principes et méthodes

IEC 61360-1:2017-07(en-fr)

IEC Central Office
3, rue de Varembé
CH-1211 Geneva 20
Switzerland

Tel.: +41 22 919 02 11
Fax: +41 22 919 03 00
info@iec.ch
www.iec.ch

About the IEC
The International Electrotechnical Commission (IEC) is the leading global organization that prepares and publishes International Standards for all electrical, electronic and related technologies.

About IEC publications
The technical content of IEC publications is kept under constant review by the IEC. Please make sure that you have the latest edition, a corrigenda or an amendment might have been published.

IEC Catalogue - webstore.iec.ch/catalogue
The stand-alone application for consulting the entire bibliographical information on IEC International Standards, Technical Specifications, Technical Reports and other documents. Available for PC, Mac OS, Android Tablets and iPad.

IEC publications search - www.iec.ch/searchpub
The advanced search enables to find IEC publications by a variety of criteria (reference number, text, technical committee,…). It also gives information on projects, replaced and withdrawn publications.

IEC Just Published - webstore.iec.ch/justpublished
Stay up to date on all new IEC publications. Just Published details all new publications released. Available online and also once a month by email.

Electropedia - www.electropedia.org
The world's leading online dictionary of electronic and electrical terms containing 20 000 terms and definitions in English and French, with equivalent terms in 16 additional languages. Also known as the International Electrotechnical Vocabulary (IEV) online.

IEC Glossary - std.iec.ch/glossary
65 000 electrotechnical terminology entries in English and French extracted from the Terms and Definitions clause of IEC publications issued since 2002. Some entries have been collected from earlier publications of IEC TC 37, 77, 86 and CISPR.

IEC Customer Service Centre - webstore.iec.ch/csc
If you wish to give us your feedback on this publication or need further assistance, please contact the Customer Service Centre: csc@iec.ch.

A propos de l'IEC
La Commission Electrotechnique Internationale (IEC) est la première organisation mondiale qui élabore et publie des Normes internationales pour tout ce qui a trait à l'électricité, à l'électronique et aux technologies apparentées.

A propos des publications IEC
Le contenu technique des publications IEC est constamment revu. Veuillez vous assurer que vous possédez l'édition la plus récente, un corrigendum ou amendement peut avoir été publié.

Catalogue IEC - webstore.iec.ch/catalogue
Application autonome pour consulter tous les renseignements bibliographiques sur les Normes internationales, Spécifications techniques, Rapports techniques et autres documents de l'IEC. Disponible pour PC, Mac OS, tablettes Android et iPad.

Recherche de publications IEC - www.iec.ch/searchpub
La recherche avancée permet de trouver des publications IEC en utilisant différents critères (numéro de référence, texte, comité d'études,…). Elle donne aussi des informations sur les projets et les publications remplacées ou retirées.

IEC Just Published - webstore.iec.ch/justpublished
Restez informé sur les nouvelles publications IEC. Just Published détaille les nouvelles publications parues. Disponible en ligne et aussi une fois par mois par email.

Electropedia - www.electropedia.org
Le premier dictionnaire en ligne de termes électroniques et électriques. Il contient 20 000 termes et définitions en anglais et en français, ainsi que les termes équivalents dans 16 langues additionnelles. Egalement appelé Vocabulaire Electrotechnique International (IEV) en ligne.

Glossaire IEC - std.iec.ch/glossary
65 000 entrées terminologiques électrotechniques, en anglais et en français, extraites des articles Termes et Définitions des publications IEC parues depuis 2002. Plus certaines entrées antérieures extraites des publications des CE 37, 77, 86 et CISPR de l'IEC.

Service Clients - webstore.iec.ch/csc
Si vous désirez nous donner des commentaires sur cette publication ou si vous avez des questions contactez-nous: csc@iec.ch.

IEC 61360-1

Edition 4.0 2017-07

INTERNATIONAL STANDARD

NORME INTERNATIONALE

colour inside

Standard data element types with associated classification scheme – Part 1: Definitions – Principles and methods

Types normalisés d'éléments de données avec plan de classification – Partie 1: Définitions – Principes et méthodes

INTERNATIONAL ELECTROTECHNICAL COMMISSION

COMMISSION ELECTROTECHNIQUE INTERNATIONALE

ICS 31.020

ISBN 978-2-8322-4581-1

Warning! Make sure that you obtained this publication from an authorized distributor.
Attention! Veuillez vous assurer que vous avez obtenu cette publication via un distributeur agréé.

CONTENTS

INTERNATIONAL ELECTROTECHNICAL COMMISSION

____

STANDARD DATA ELEMENT TYPES WITH ASSOCIATED CLASSIFICATION SCHEME –

Part 1: Definitions – Principles and methods

FOREWORD

1) The International Electrotechnical Commission (IEC) is a worldwide organization for standardization comprising all national electrotechnical committees (IEC National Committees). The object of IEC is to promote international co-operation on all questions concerning standardization in the electrical and electronic fields. To this end and in addition to other activities, IEC publishes International Standards, Technical Specifications, Technical Reports, Publicly Available Specifications (PAS) and Guides (hereafter referred to as “IEC Publication(s)”). Their preparation is entrusted to technical committees; any IEC National Committee interested in the subject dealt with may participate in this preparatory work. International, governmental and non-governmental organizations liaising with the IEC also participate in this preparation. IEC collaborates closely with the International Organization for Standardization (ISO) in accordance with conditions determined by agreement between the two organizations.
2) The formal decisions or agreements of IEC on technical matters express, as nearly as possible, an international consensus of opinion on the relevant subjects since each technical committee has representation from all interested IEC National Committees.
3) IEC Publications have the form of recommendations for international use and are accepted by IEC National Committees in that sense. While all reasonable efforts are made to ensure that the technical content of IEC Publications is accurate, IEC cannot be held responsible for the way in which they are used or for any misinterpretation by any end user.
4) In order to promote international uniformity, IEC National Committees undertake to apply IEC Publications transparently to the maximum extent possible in their national and regional publications. Any divergence between any IEC Publication and the corresponding national or regional publication shall be clearly indicated in the latter.
5) IEC itself does not provide any attestation of conformity. Independent certification bodies provide conformity assessment services and, in some areas, access to IEC marks of conformity. IEC is not responsible for any services carried out by independent certification bodies.
6) All users should ensure that they have the latest edition of this publication.
7) No liability shall attach to IEC or its directors, employees, servants or agents including individual experts and members of its technical committees and IEC National Committees for any personal injury, property damage or other damage of any nature whatsoever, whether direct or indirect, or for costs (including legal fees) and expenses arising out of the publication, use of, or reliance upon, this IEC Publication or any other IEC Publications.
8) Attention is drawn to the Normative references cited in this publication. Use of the referenced publications is indispensable for the correct application of this publication.
9) Attention is drawn to the possibility that some of the elements of this IEC Publication may be the subject of patent rights. IEC shall not be held responsible for identifying any or all such patent rights.

International Standard IEC 61360-1 has been prepared by subcommittee 3D: Product properties and classes and their identification, of IEC technical committee 3: Information structures and elements, identification and marking principles, documentation and graphical symbols.

This fourth edition cancels and replaces the third edition published in 2009. This edition constitutes a technical revision.

This edition includes the following significant technical changes with respect to the previous edition:

– support of advanced constructs such as
  - conditions and constraints,
  - blocks,

  - cardinality,
  - polymorphism,
  - generic and restricted enumerations, and
  - mapping;
- extended list of data types;
- harmonization with IEC 62656-1;
- support of IEC TS 62720 and of coded units;
- harmonization of semantic and administrative data among the various information objects;
- use of UML for data modelling;
- enhanced definitions and descriptions;
- introduction of examples of higher level constructs such as block, cardinality, or polymorphism as guidance for the user of the IEC 61360 series.

The text of this International Standard is based on the following documents:

FDIS	Report on voting
3D/295/FDIS	3D/298/RVD

Full information on the voting for the approval of this International Standard can be found in the report on voting indicated in the above table.

This document has been drafted in accordance with the ISO/IEC Directives, Part 2.

A list of all parts in the IEC 61360 series, published under the general title *Standard data element types with associated classification scheme,* can be found on the IEC website.

Future standards in this series will carry the new general title as cited above. Titles of existing standards in this series will be updated at the time of the next edition.

The committee has decided that the contents of this document will remain unchanged until the stability date indicated on the IEC website under "http://webstore.iec.ch" in the data related to the specific document. At this date, the document will be

- reconfirmed,
- withdrawn,
- replaced by a revised edition, or
- amended.

IMPORTANT – The 'colour inside' logo on the cover page of this publication indicates that it contains colours which are considered to be useful for the correct understanding of its contents. Users should therefore print this document using a colour printer.

INTRODUCTION

The IEC 61360 series as a whole specifies a general purpose dictionary of technical terms covering the field of electrotechnology, electronics and related domains. The dictionary is specified in a computer-sensible form as a reference dictionary. By using the dictionary , applications can interact and share data in an unambiguous way free from semantic uncertainties.

This document addresses domain engineers and provides a detailed introduction to the structure of the dictionary and its uses from the viewpoint of the dictionary provider as well as from the viewpoint of the user of the dictionary. IEC 61360-2 specifies the detailed dictionary data model and IEC 61360-6 stipulates quality criteria for the content of the dictionary.

Referencing to a common dictionary is advantageous in all cases where product information has to be transferred in an unambiguous way. Use cases include catalogues, ordering processes, product information contained in specifications or contracts.

The International Electrotechnical Commission has set up a technical dictionary for the use in the electrotechnical and electronic domain which is maintained by SC 3D. This dictionary is called IEC Common Data Dictionary (IEC CDD) and can be accessed on the following IEC web page: http://std.iec.ch/iec61360.

Dictionaries should not be confused with catalogues or master data collections; these make use of dictionary objects. In catalogues or master data collections, values are assigned to instances of dictionary objects. Thus they build upon dictionaries. Consequently, dictionaries are normally exchanged in advance of any catalogue or master data.

Closely associated with this document is IEC 61360-2, which contains the information model using the EXPRESS modelling language. In this model, the definition and structure of IEC 61360-1 is formalized and presented in a computer-sensible form.

This document is largely compliant with IEC 61360-2:2012 and ISO 13584-42:2010. However, practical use has shown the necessity to selectively extend the data model. Thus, this fourth edition of IEC 61360-1 extends IEC 61360-2 in both semantics and syntax and introduces additional constructs available from IEC 62656-1 for practical benefits. For example, constructs such as enumerations of real numbers or relation objects do not exist in IEC 61360-2, but are needed in many fields of engineering.

IEC 62656-1 provides interfaces to IEC CDD. Thus, change requests to IEC CDD need to be formed according to the interface specification available from the latter standard.

In some cases it can be difficult to see whether words represent names of information objects or if just everyday items are addressed, e.g. by the term "property".

For this reason, the following typographic rules are used in this document:

– bold, upper case first letter: name of a UML class, e.g. "**Property**";
– bold, lower case first letter: name of a UML attribute or UML association, e.g. "**revision_number**";
– normal font (i.e. not bold), lower case first letter: floating text, e.g. "property".

UML information object names should be treated as constants and should not be translated in other languages.

STANDARD DATA ELEMENT TYPES WITH ASSOCIATED CLASSIFICATION SCHEME –

Part 1: Definitions – Principles and methods

1 Scope

This part of IEC 61360 specifies principles for the definition of the properties and associated attributes and explains the methods for representing verbally defined concepts with appropriate data constructs available from IEC 61360-2. It also specifies principles for establishing a hierarchy of classification from a collection of classes, each of which represents a technical concept in the electrotechnical domain or a domain related to electrotechnology.

The use of this document facilitates the exchange of technical data through a defined structure for the information to be exchanged in a computer-sensible form. Each property to be exchanged has an unambiguously defined meaning and consistent naming, where relevant a defined value list, a prescribed format and defined units of measure for all quantitative values. There is also provision for:

a) control of changes to definitions of the properties through version and revision numbers;
b) inclusion of notes and remarks to clarify and help in the application of the definitions;
c) indication of the sources of definitions and value lists;
d) associated figures and formulae.

NOTE IEC TCs and SCs, or other organizations can take this document as a basis for the development of their own dictionaries.

Out of scope of this document are subjects concerning the information technology infrastructure such as:

– security;
– database locking mechanisms;
– access rights management.

2 Normative references

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

IEC 62656-1:2014, *Standardized product ontology register and transfer by spreadsheets – Part 1: Logical structure for data parcels*

IEC TS 62720, *Identification of units of measurement for computer-based processing*

IEC 80000 (all parts), *Quantities and units*

ISO/IEC 646, *Information technology – ISO 7-bit coded character set for information interchange*

ISO/IEC 10646, *Information technology – Universal Multiple-Octet Coded Character set (UCS)*

ISO/IEC 11179-3:, *Information technology – Metadata registries (MDR) – Part 3: Registry metamodel and basic attributes*

ISO/IEC 19505-1, *Information technology – Object Management Group Unified Modeling Language (OMG UML) – Part 1: Infrastructure*

ISO 639-1, *Codes for the representation of names of languages – Part 1: Alpha-2 code*

ISO 2382 (all parts), *Information processing systems – Vocabulary*

ISO 3166 (all parts), *Codes for the representation of names of countries and their subdivisions*

ISO 8601:2004, *Data elements and interchange formats – Information interchange – Representation of dates and times*

ISO 10303-11, *Industrial automation systems and integration – Product data representation and exchange – Part 11: Description methods: The EXPRESS language reference manual*

ISO 13584-26:2000, *Industrial automation systems and integration – Parts library – Part 26: Logical resource: Information supplier identification*

ISO 13584-42:2010, *Industrial automation systems and integration – Parts library – Part 42: Description methodology: Methodology for structuring part families*

ISO/TS 29002-5, *Industrial automation systems and integration – Exchange of characteristic data – Part 5: Identification scheme*

ISO 80000 (all parts), *Quantities and units*

IETF RFC 2045: N. Freed and N. Borenstein, *Multipurpose Internet Mail Extensions (MIME) Part One: Format of Internet Message Bodies* [online], November 1996 [viewed 2016-07-01]. Available at http://www.ietf.org/rfc/rfc2045.txt

IETF RFC 2046: N. Freed and N. Borenstein, *Multipurpose Internet Mail Extensions (MIME) Part Two: Media Types* [online], November 1996 [viewed 2016-07-01]. Available at http://www.ietf.org/rfc/rfc2046.txt

IETF RFC 2047: K. Moore, *MIME (Multipurpose Internet Mail Extensions) Part Three: Message Header Extensions for Non-ASCII Text* [online], November 1996 [viewed 2016-07-01]. Available at http://www.ietf.org/rfc/rfc2047.txt

3 Terms, definitions and abbreviated terms

3.1 Terms and definitions

For the purposes of this document, the following terms and definitions apply

ISO and IEC maintain terminological databases for use in standardization at the following addresses:

- IEC Electropedia: available at http://www.electropedia.org/
- ISO Online browsing platform: available at http://www.iso.org/obp

3.1.1
applicable property
property necessarily possessed by each part that is member of a characterization class

Note 1 to entry: Each part that is member of a characterization class possesses an aspect corresponding to each applicable property of this characterization class.

Note 2 to entry: The above definition is conceptual, there is no requirement that all the applicable properties of a class should be used for describing each part of this class at the data model level.

Note 3 to entry: All the applicable properties of a superclass are also applicable properties for the subclasses of this superclass.

Note 4 to entry: Only properties defined or inherited as visible and imported properties of a class can be applicable properties.

Note 5 to entry: To facilitate integration of component libraries and electronic catalogues based on ISO 13584-24:2003 and ISO 13584-25, these parts of ISO 13584 request that only properties that are applicable to a class be used to characterize their instances in component libraries and electronic catalogues.

[SOURCE: IEC 61360-2:2012, 3.2]

3.1.2
attribute
data element for the computer-sensible description of a property, a relation or a class

EXAMPLE The name of a property, the code of a class, the measure unit in which values of a property are provided.

Note 1 to entry: An attribute describes only one single detail of a property, of a class or of a relation.

[SOURCE: ISO/IEC Guide 77-2:2008, 2.2]

3.1.3
cardinality
pattern defining the number of times a concept reoccurs within a description

[SOURCE: IEC 61987-10:2009, 3.1.5, modified — Notes 1 to 5 are omitted.]

3.1.4
characteristic
distinguishing feature

Note 1 to entry: A characteristic can be inherent or assigned.

Note 2 to entry: A characteristic can be qualitative or quantitative.

[SOURCE: ISO 22274:2013, 3.3, modified — Note 4 and the Example are omitted.]

3.1.5
classification
systematic division of a set of items into subsets according to their difference in some predetermined characteristics

3.1.6
class
abstraction of a set of similar products

Note 1 to entry: A product that complies with the abstraction defined by a class is called a class member.

Note 2 to entry: A class is an intentional concept that can take different extensional meanings in different contexts.

EXAMPLE The set of products used by a particular enterprise and the set of all ISO-standardized products are two examples of contexts. In these two contexts (the particular enterprise and ISO), the set of products that are considered as members of the *single ball bearing* class can be different, in particular because employees of each enterprise ignore a number of existing single ball bearing products.

Note 3 to entry: Classes are structured by class inclusion relationships.

Note 4 to entry: A class of products is a general concept as defined in ISO 1087-1. Thus, it is advisable that the rules defined in ISO 704 be used for defining the designation and definition attributes of classes of products.

Note 5 to entry: In the context of the ISO 13584 series, a class is either a characterization class, associated with properties and usable for characterizing products, or a categorization class, not associated with properties and not usable for characterizing products.

[SOURCE: IEC 61360-2:2012, 3.6]

3.1.7
classifying property
property applicable for a particular class, having a value list whose values define the subclasses of the class

3.1.8
concept
unit of knowledge created by a unique combination of characteristics

[SOURCE: ISO 22274:2013, 3.7]

3.1.9
component
industrial product which serves a specific function or functions, which is intended for use in a higher order assembled product

Note 1 to entry: A component is in its context considered not to be decomposable or physically divisible. In another context, such as a service shop, a component can be decomposable.

3.1.10
computer-sensible form
specific representation of information allowing processing of the information content by means of an electronic computer

3.1.11
drawing
illustration used for explanatory purposes

3.1.12
entity
class of information defined by common properties

[SOURCE: ISO 10303-11:2004, 3.3.6]

3.1.13
enumeration
list of named constants called enumerators, each numerator name in the enumeration being unambiguous

3.1.14
feature
aspect of an item that can be captured by a class structure and set of properties and that cannot exist independently of the item

[SOURCE: ISO 13584-24:2003, 3.41, modified – Examples 1 and 2 are omitted.]

3.1.15
information object
well-defined piece of information, definition, or specification which requires a name in order to identify its use in communication

3.1.16
item
thing that can be captured by a structure of class or by a structure of property

Note 1 to entry: Items are considered to reach from a component to assemblies up to systems.

[SOURCE: IEC 62656-1:2014, 3.23]

3.1.17
limiting value
greatest or smallest admissible value of a quantity in a specification of a component, device, equipment, or system beyond which it incurs damage resulting in permanent unwanted changes of functional or physical characteristics influencing its performance

3.1.18
minimum value
min
lower bound of a range of values in which the said value is meaningful

EXAMPLE Lowest value specified of a quantity, established for a specified set of operating conditions at which a component, device, equipment, or system can operate and perform according to specified requirements.

Note 1 to entry: Additional information about the nature of the value can be obtained from the definition of the **Property** information object to which the value belongs.

3.1.19
maximum value
max
upper bound of a range of values in which the said value is meaningful

EXAMPLE Highest value specified of a quantity, established for a specified set of operating conditions at which a component, device, equipment, or system can operate and perform according to specified requirements

Note 1 to entry: Additional information about the nature of the value can be obtained from the definition of the **Property** information object to which the value belongs.

3.1.20
nominal value
nom
value of a quantity used to designate or identify an item with its value, and not necessarily corresponding to the real value of the property

Note 1 to entry: Additional information about the nature of the value can be obtained from the definition of the **Property** information object to which the value belongs.

3.1.21
non-quantitative property
property that identifies or describes an object by means of codes, abbreviations, names, references or descriptions

EXAMPLE Typical information content of non-quantitative properties is items such as codes, abbreviations, names, references, or descriptions.

[SOURCE: IEC 61360-2:2012, 3.28, modified – "data element type" is replaced by "property" in the term and definition.]

3.1.22
polymorphism
pattern that allows substitution of a single concept in the same context by a different more specific (specialized) concept

Note 1 to entry: A specialized polymorphic block can replace a more generic one in the same context.

Note 2 to entry: A polymorphic operator (control property) can act in selecting between various specializations.

[SOURCE: IEC 61987-10:2009, 3.1.21]

3.1.23
product
result of labour or of a natural or industrial process

3.1.24
property
data element type
defined parameter suitable for the description and differentiation of objects

Note 1 to entry: A property describes one characteristic of a given object.

Note 2 to entry: A property can have attributes such as code, version, and revision.

Note 3 to entry: The specification of a property can include predefined choices of values.

[SOURCE: ISO/IEC Guide 77-2:2008, 2.18, modified — In the definition, "products" is replaced by "objects". Note 4 is omitted]

3.1.25
quantitative property
property with a numerical value representing a physical quantity, a quantity of information or a count of objects

[SOURCE: IEC 61360-2:2012, 3.40, modified – "data element type" is replaced by "property" in the term and definition.]

3.1.26
quantity
property of a phenomenon, body, or substance, where the property has a magnitude that can be expressed by means of a number and a reference

Note 1 to entry: The generic concept “quantity” can be divided into several levels of specific concepts, as shown in Table 1. The left hand side of the table shows specific concepts under “quantity”. These are generic concepts for the individual quantities in the right hand column.

Table 1 – Examples of generic concepts for individual quantities

Generic concepts for individual quantities		Individual quantities
length, l	radius, r	radius of circle A, r_A or r(A)
	wavelength, λ	wavelength of the sodium D radiation, λ_D or λ(D; Na)
energy, E	kinetic energy, T	kinetic energy of particle i in a given system, T_i
	heat, Q	heat of vaporization of sample i of water, Q_i
electric charge, Q		electric charge of the proton, e
electric resistance, R		electric resistance of resistor i in a given circuit, R_i
amount-of-substance concentration of substance B, c_B		amount-of-substance concentration of ethanol in wine sample i, $c_i(C_2H_5OH)$
number concentration of substance B, C_B		number concentration of erythrocytes in blood sample i, C(Erys; B_i)
Rockwell C hardness (150 kg load), HRC(150 kg)		Rockwell C hardness of steel sample i, HRC_i(150 kg)

Note 2 to entry: A reference can be a measurement unit, a measurement procedure, a reference material, or a combination of them.

Note 3 to entry: Symbols for quantities are given in the IEC 80000 and ISO 80000 series. The symbols for quantities are written in italics. A given symbol can indicate different quantities.

Note 4 to entry: A quantity as defined here is a scalar. However, a vector or a tensor, the components of which are quantities, is also considered to be a quantity.

Note 5 to entry: The concept "quantity" can be generically divided into, for example, "physical quantity", "chemical quantity", and "biological quantity", or "base quantity" and "derived quantity".

Note 6 to entry: Adapted from ISO/IEC Guide 99:2007, definition 1.1, in which there is an additional note.

[SOURCE: ISO 80000-1:2009, 3.1]

3.1.27
relation
aspect or quality that connects two or more things or parts as being or belonging together

3.1.28
subclass
class that is one step below another class in a class inclusion hierarchy

Note 1 to entry: In the common ISO 13584/IEC 61360 dictionary model, class inclusion hierarchies are defined by the is-a relationship. They can also be established by the case-of relationships.

[SOURCE: IEC 61360-2:2012, 3.43]

3.1.29
superclass
class that is one step above another class in a class inclusion hierarchy

Note 1 to entry: In the common ISO 13584/IEC 61360 dictionary model, class inclusion hierarchies are defined by the is-a relationship. They can also be established by the case-of relationships.

Note 2 to entry: In the common ISO 13584/IEC 61360 dictionary model, a class has at most one superclass specified by means of an is-a relationship.

[SOURCE: IEC 61360-2:2012, 3.44]

3.1.30
typical value
typ
value of a quantity used as a representative value or commonly observable value

EXAMPLE Typical value specified of a quantity, established for a specified set of operating conditions at which an item can operate and perform according to specified requirements.

Note 1 to entry: Additional information about the nature of the value can be obtained from the definition of the **Property** information object to which the value belongs.

3.1.31
visible property
property that has a definition meaningful in the scope of a given characterization class, but that does not necessarily apply to the various products belonging to this class

Note 1 to entry: Meaningful in the scope of a given characterization class means that a human observer is able to determine, for any product of the characterization class, whether the property applies, and, if it applies, to which product aspect it corresponds.

Note 2 to entry: The concept of a visible property allows sharing the definition of a property among product characterization classes where this property does not necessarily apply.

EXAMPLE The non-threaded length property is meaningful for any class of screw but it applies only to those screws that have a non-threaded part. It can be defined as visible at the screw level, while becoming applicable only in some subclasses.

Note 3 to entry: All the visible properties of a superclass that is a product characterization class are also visible properties for its subclasses.

Note 4 to entry: To facilitate integration of component libraries and electronic catalogues based on ISO 13584-24:2003 and ISO 13584-25, these parts of ISO 13584 request that only properties that are applicable to a class be used to characterize their instances in component libraries and electronic catalogues.

Note 5 to entry: This definition of a visible property supersedes the previous definition from ISO 13584-24:2003 that was the following: “a property that is defined for some class of products and that does not necessarily apply to the different products of this class of products”.

[SOURCE: IEC 61360-2:2012, 3.46]

3.2 Abbreviated terms

DI	Data Identifier
HTML	Hypertext Mark-up Language
ICID	International Concept Identifier
IEC CDD	IEC Common Data Dictionary
IRDI	International Registration Data Identifier
RAI	Registration Authority Identifier
UML	Unified Modelling Language
URI	Uniform Resource Identifier
URL	Uniform Resource Locator

4 Foundation for the concepts of IEC 61360 standard dictionaries

In the IEC 61360 series, the basic concepts underlying the full data model are those of "dictionary", "class", and "property".

The concept of a product is captured by entities called "classes" and its characteristics are captured by entities called "properties". A hierarchy of classes in this document is an ordering of classes according to the commonalities of the properties in the classes.

The following clauses provide details on these concepts and their interrelations.

To show the concepts Unified Modelling Language (UML, see ISO/IEC 19505-1) class diagrams are used in Clauses 6 to 23. Clause 24 uses a spreadsheet-based presentation introduced by IEC 62656-1:2014 for representing structures containing instances.

NOTE 1 The UML models are for explanatory purposes; a detailed data model for implementation is given in IEC 61360-2.

Figure 1 shows a simplified overview model of the IEC 61360-1 information objects and their interrelations.

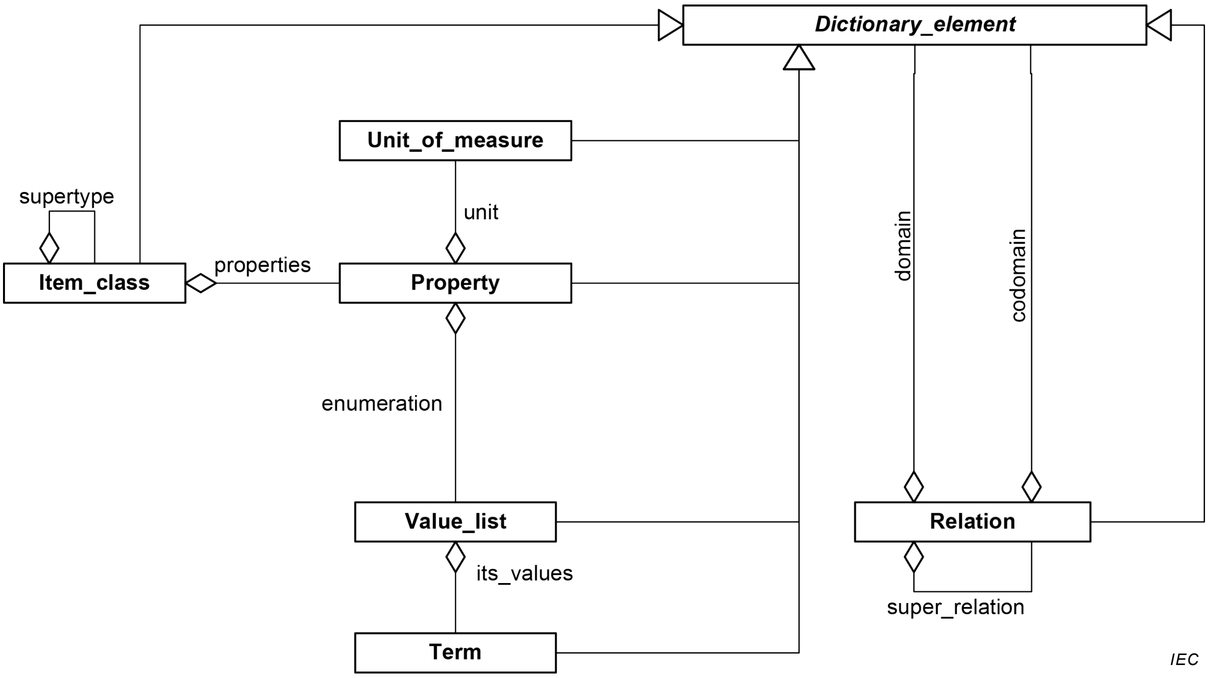

Figure 1 – Simplified model of IEC 61360-1 (UML class diagram)

NOTE 2 The association enumeration of Figure 1 is implemented via the enumeration data types.

The IEC 61360 dictionary model has a single inheritance model. However, from time to time, some properties defined in another class which resides in other branches of specialization hierarchy become necessary and pertinent.

In this case, the model offers the case-of mechanism to import properties of a fixed version (see 24.3). Once imported into a class, such properties become applicable in the class and inherited into its subclasses as applicable properties. The importation of properties is achieved by listing the properties to be imported in the “imported_properties” attribute of the case-of class.

Note that once a case-of relationship is installed, the version of the imported property is fixed, and even if the definition of the property is changed in the source class, the imported property does not change.

See Annex A for the detailed UML class diagrams of IEC 61360-1. Annex H gives a short introduction of the used UML constructs.

Annex C provides advice on preparation of new classes and properties.

5 Dictionary identification

5.1 General

The IEC Common Data Dictionary (IEC CDD) contains a collection of classes and properties. All information objects in a dictionary are uniquely identified including a version identifier as a record of significant changes to a dictionary object. This enables a version control for each dictionary object. And the identification of a dictionary object is therefore not dependent on a publication of the whole dictionary at a particular point in time.

For use of the dictionary content in communication it can be helpful to identify the complete set of dictionary items with a reference. For this reason the IEC CDD as a whole is identified by the following attributes given in 5.2 to 5.6.

5.2 Dictionary_supplier

Attribute name: **dictionary_supplier**

Attribute definition: identification of any organization authorized to register dictionary items

Description: The identifier is specified in ISO 13584-26:2000. The source identification for the IEC 61360-4 reference dictionary is: "0112/2///61360_4".

Other sources may require different identifications.

NOTE 1 The source identification of the standard does not include an edition number contrary to what ISO 13584-26:2000 has defined.

NOTE 2 The hyphen "-" in "61360-4" is replaced by an underscore "_".

EXAMPLE "0112/2///61987" is the source identification for data of IEC 61987 series.

Obligation: mandatory

Character type of values: String constant as specified

NOTE 3 The slash “/” separator is used in a physical file in accordance with ISO 10303-21. In other environments such as XML other separators can be used.

5.3 Code

Attribute name: **code**

Attribute definition: unique identifier of the dictionary item

Description: The content of **code** for the IEC Common Data Dictionary shall be: "IEC CDD".

Obligation: mandatory

Character type of values: String constant as specified

5.4 Version_number

Attribute name: **version_number**

Attribute definition: number used to control the versions of the IEC Common Data Dictionary

Description: The **version_number** of the dictionary should consist of 6 digits. Although the common ISO/IEC EXPRESS information model allows more digits, for the dictionary to be registered into IEC CDD or to be compatible with IEC 61360 series only 6 digits should be used. For specific purposes, version numbers with more digits may be used. Consecutive version numbers starting from "000001" shall be issued in ascending order.

A new version of the dictionary may be generated if at least one class or property is added or when changes are made affecting their version number. When a new version of the dictionary is issued, this data set contains all changes that happened since publication of the last version.

NOTE Intermediate changes of the dictionary can be traced by checking the attribute **date_of_current_version**.

Obligation: mandatory

Character type of values: Digits "0" through "9".

5.5 Date_of_current_version

Attribute name: **date_of_current_version**

Attribute definition: date associated with the present content of the IEC Common Data Dictionary

Description: The date information of the dictionary shall consist of 10 digits and shall follow the format "YYYY-MM-DD" as specified in ISO 8601:2004.

Whenever the content of the dictionary is modified, the value of the attribute shall be updated.

EXAMPLE "1994-03-21".

Obligation: mandatory

Character type of values: Digits "0" through "9" and "-".

5.6 Revision_number

Attribute name: **revision_number**

Attribute definition: number used for administrative control of the IEC Common Data Dictionary

Description:	The **revision_number** of the dictionary should consist of 3 digits. Although the common ISO/IEC EXPRESS information model allows more digits, for the dictionary to be registered into IEC CDD or to be compatible with IEC 61360 series only 3 digits should be used. For specific purposes, revision numbers with more digits may be used. Consecutive revision numbers starting from "000" shall be issued in ascending order. Only one **revision_number** of the dictionary, unique by its identifier, is current at any moment. A new **revision_number** of the dictionary shall be generated if an attribute of a dictionary object is changed which neither affects the use, nor the meaning of the dictionary objects. EXAMPLE Editorial changes such as correction of typing errors. The content of **revision_number** shall be reset to the starting value "1" when the **version_number** is incremented
Obligation:	mandatory
Character type of values:	Digits "0" through "9".

6 Property

6.1 Overview

A property is a characteristic of an object such as a component (see 3.1.24, Note 1). A property, when used, always has an associated value, such as a numerical value together with a unit, or a qualitative value taken from a predefined list.

In Clause 6, the various attributes of properties as encountered in the specifications are explained. For a list of these attributes, see Table 2. These attributes are related to identification, description, value of characteristics and to its relationships.

When specifying properties and their associated data type care, should be taken that the anticipated value is meaningful. In particular, the use of BOOLEAN_TYPE values may lead to deficient specifications if the meaning of the values "TRUE" and "FALSE" is not properly specified. In many cases the use of a value list instead of Boolean values provides more precise information and should be preferred.

Table 2 – List of attributes of Property information objects as defined in IEC 61360-1 and their equivalent in IEC 61360-2

Attribute names in IEC 61360-1	Information objects in IEC 61360-2	Subclause number
code	code	7.3
code (of value)	value_code	7.3
code (of value list)	value_code	7.3
code_of_alternative_unit	[a]	6.9
code_of_unit	[a]	6.11
code_of_list_of_units	[a]	6.10
condition_property	condition_DET	11.2
data_element_type_class	det_classification	6.3
data_type	data_type	10.4
date_of_current_translation_revision	[a]	15.2
definition	definition	8.3
definition (of value)	preferred_name of meaning	8.3
definition (of value list)	preferred_name of meaning	8.3
dependent_condition_property	[a]	12
dependent_property	dependent_P_DET	13
drawing_reference	figure	6.13
formula	formula	6.5
language_code	[a]	15.3
non_dependent_property	non_dependent_P_DET	14
note	note	8.4
obsolete_date	is_deprecated/ date_of_current_version	9.3
preferred_letter_symbol	preferred_symbol	6.6
preferred_name	preferred_name	7.4
proposal_date	[a]	9.4
published_by	[a]	9.6
published_in	[a]	9.5
referenced_class_identifier	domain of class_instance_type	10.6
remark	remark	8.5
responsible_committee	[a]	9.7
responsible_translator	[a]	15.4
responsible_translator_coded	[a]	15.5
revision_number	revision	7.5
revision_released_on	date_of_current_revision	9.8
short_name	short_name	7.6
source_document_of_definition	source_doc_of_definition	8.6
source_document_of_definition (of value list)	source_doc_of_value_domain	8.6
status_level	[a]	9.9
synonymous_letter_symbol	synonymous_symbol	6.7
synonymous_name	synonymous_name	7.7
translation_revision	[a]	15.6
unit_of_measure	unit	10.7
value_format	value_format	10.8
value_list	value_domain	16

Attribute names in IEC 61360-1	Information objects in IEC 61360-2	Subclause number
version_initiated_on	date_of_original_definition	9.10
version_number	version	7.8
version_released_on	date_of_current_version	9.11
[a] Not defined in IEC 61360-2:2012.		

Based on the principles as described in ISO/IEC 11179-3, the attributes are divided into four main groups:

– identification related attributes;
– semantics related attributes;
– value related attributes;
– relationship attributes.

Figure 2 shows the overview model of the module "Characteristic" using UML as presentation technique.

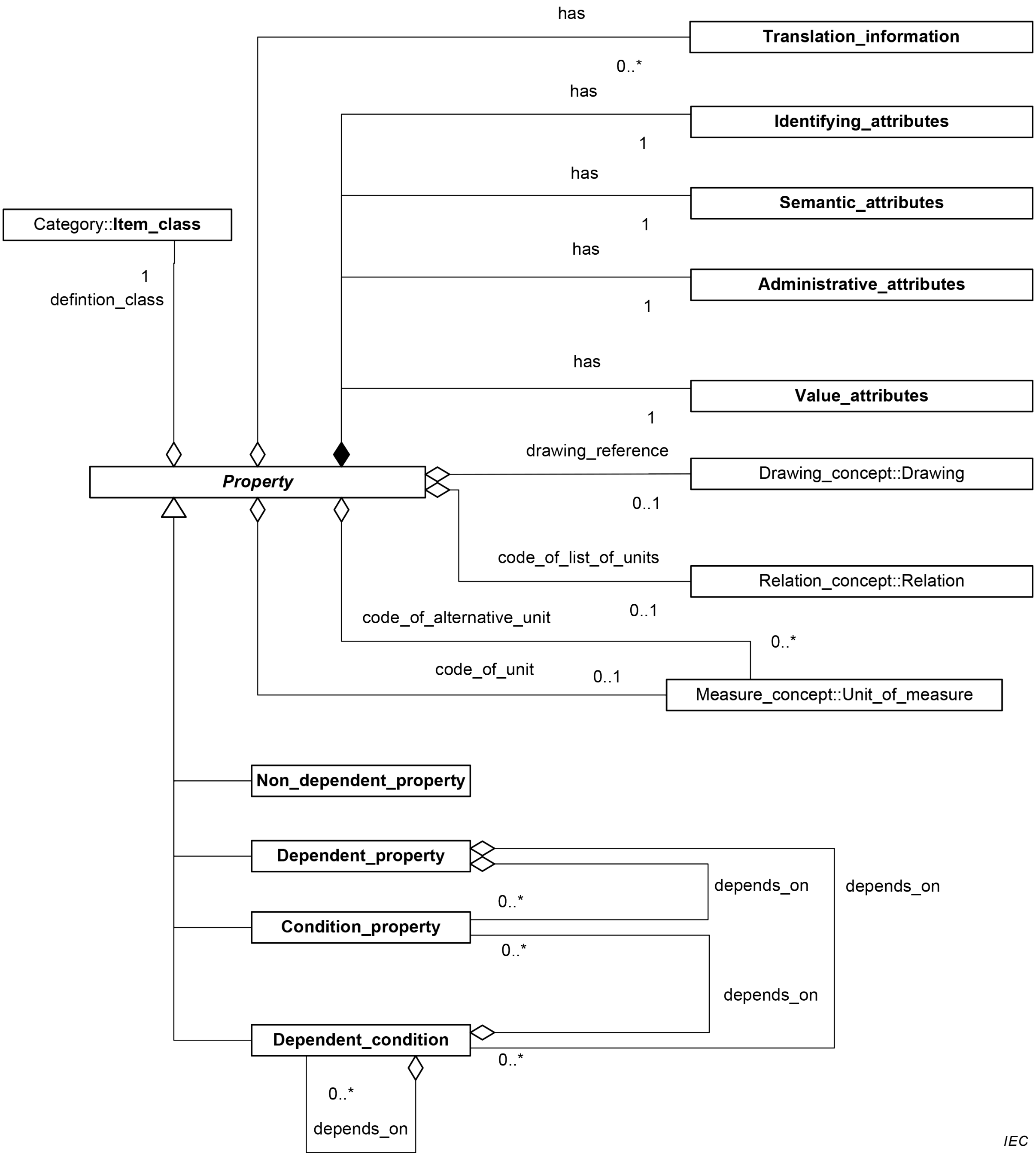

Figure 2 – Overview model for Characteristic (UML class diagram)

6.2 Specification of information object

Name: **Property**

Definition: information object that characterizes the **Item_class**(es) to which it is associated

Description:

Figure 3 shows the detailed model of the information object "**Property**" using UML as presentation technique.

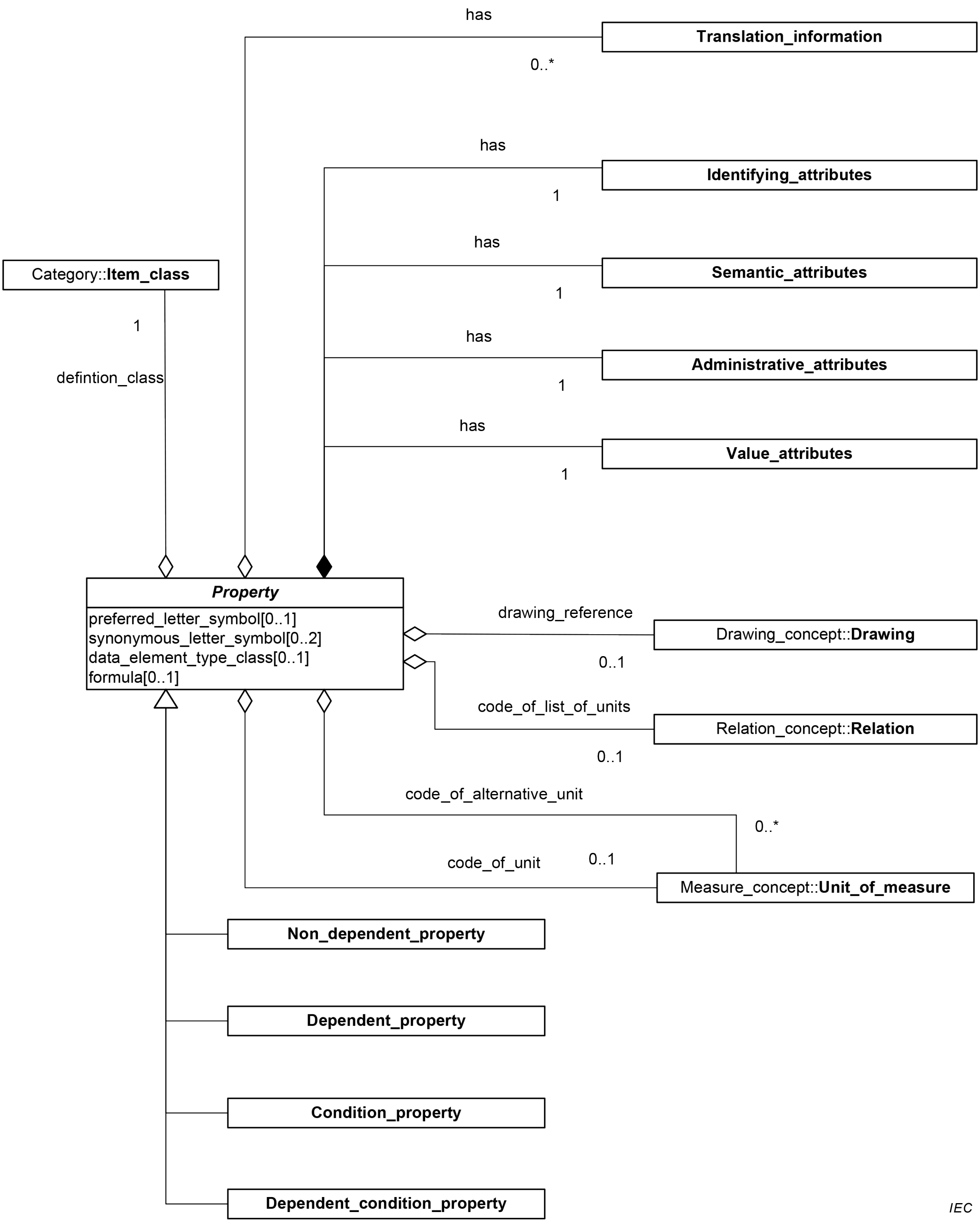

Figure 3 – Property (UML class diagram)

6.3 Data_element_type_class

Attribute name:	**data_element_type_class**
Attribute definition:	classification of similar property types
Description:	Property classification is described in detail in Annex B.

Obligation:	optional
Character type of values:	See Clause 26.

6.4 Depends_on

Attribute name:	**depends_on**
Attribute definition:	set of one or more code(s) of **Property** information objects (see 7.3) acting as conditions on which the **Property** in question depends
Description:	See also 24.2.
Obligation:	conditional
Condition:	shall have a value if **data_element_type_class** has one of the values DEPENDENT_P_DET DEPENDENT_C_DET
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

6.5 Formula

Attribute name:	**formula**
Attribute definition:	rule or statement in mathematical form expressing semantics of a **Property**
Description:	The content of **formula** shall not change any essential information of the meaning of the definition. The graphical representation of a formula shall be stored as a file in a general available format. NOTE 1 MathML (see W3C Recommendation 10:2014) is recommended as language to present mathematical formulas.
Obligation:	optional
Character type of values:	See Clause 26.

6.6 Preferred_letter_symbol

Attribute name:	**preferred_letter_symbol**
Attribute definition:	mark or characters used as a sign for expressing an information object EXAMPLE 1 The characters standing for chemical elements in the periodic system (Ag = silver) or the characters standing for electrical concepts (f = frequency, T_{amb} = ambient temperature).
Description:	The content of **preferred_letter_symbol** of a **Property** shall be identical to the letter symbol in the relevant standards. EXAMPLE 2 IEC 60027 series, IEC 60747 series, IEC 60748 series, or IEC 80000 and ISO 80000 series. For additional letter symbols, the standards relevant for the item in question should be consulted. The first character of the preferred letter symbol of a **Condition_property** should be a "@".
Obligation:	optional
Character type of values:	See Clause 26.

6.7 Synonymous_letter_symbol

Attribute name:	**synonymous_letter_symbol**
Attribute definition:	alternate mark or string of characters used as a sign for describing an information object that differs from the content of **preferred_letter_symbol** but represents the same concept
Description:	Synonymous letter symbols are sometimes inevitable for historical reasons. The number of synonymous letter symbols shall be limited to two. The first character of the synonymous letter symbol of a **Condition** should be a "@".
Obligation:	optional
Character type of values:	See Clause 26.

6.8 Attributes internal to Property

Property has attributes that are grouped for better readability in the information objects

- **Identifying_attributes;**
- **Semantic_attributes;**
- **Administrative_attributes;**
- **Translation_information;**

– **Value_attributes.**

These information objects are related to **Property** by the relationship "**has**". The information objects shall be kept together with the referencing class, i.e. they are internal to the instances of **Property**.

6.9 Code_of_alternative_unit

Attribute name:	**code_of_alternative_unit**
Attribute definition:	one or more class identifier(s) of **Unit_of_measure** information objects that may be used instead of the unit specified by **code_of_unit**
Description:	The unit of measure shall be specified by using the codes specified in IEC TS 62720 or compliant unit dictionaries. NOTE 1 The unit dictionaries can be supplied in computer-sensible form or as paper. NOTE 2 The unit identifier used in **code_of_alternative_unit** attribute can resolve to a unit provided by a server for units of measure.
Obligation:	optional
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

NOTE 3 In principle the content of **code_of_alternative_unit** has the form RAI#DI##VI where RAI stands for the registration authority identifier, DI for the data identifier, and VI for the version identifier as described in ISO/IEC 11179-5.

6.10 Code_of_list_of_units

Attribute name:	**code_of_list_of_units**
Attribute definition:	class identifier specifying a **List_of_units** which offers a collection of possible units of measure
Description:	The unit of measure may be specified by using the codes specified in IEC TS 62720 or compliant unit dictionaries. NOTE 1 The unit dictionaries can be supplied in computer-sensible form or as paper. NOTE 2 The unit identifier used in **code_of_list_of_units** attribute can resolve to a unit provided by a server for units of measure.
Obligation:	optional
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

NOTE 3 In principle the content of **code_of_list_of_units** has the form RAI#DI##VI where RAI stands for the registration authority identifier, DI for the data identifier, and VI for the version identifier as described in ISO/IEC 11179-5.

6.11 Code_of_unit

Attribute name: **code_of_unit**

Attribute definition: class identifier of the preferred unit of measure

Description: The unit of measure may be specified by using the codes specified in IEC TS 62720 or compliant unit dictionaries.

NOTE 1 The unit dictionaries can be supplied in computer-sensible form or as paper.

NOTE 2 The unit identifier used in **code_of_unit** attribute can resolve to a unit provided by a server for units of measure.

Obligation: optional

Character type of values: Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

NOTE 3 In principle the content of **code_of_unit** has the form RAI#DI##VI where RAI stands for the registration authority identifier, DI for the data identifier, and VI for the version identifier as described in ISO/IEC 11179-5.

6.12 Definition_class

Attribute name: **definition_class**

Attribute definition: class in which the information object is defined

Description: The **class** referenced by **definition_class** specifies the scope. Outside of this scope the instances are invalid.

Obligation: mandatory

Character type of values: Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

6.13 Drawing_reference

Attribute name: **drawing_reference**

Attribute definition: reference to a **Drawing** illustrating the meaning of a **Property**

Obligation: optional

Character type of values: Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

7 Identifying_attributes

7.1 Overview

The **Identifying_attributes** information object collects the attributes specifying the identity of an information object.

To identify an information object uniquely a language independent combination of characters shall be used. The identifier shall consist of the six-character code, followed by a hyphen followed by the three-digit version number of the information object.

For ensuring the global uniqueness of the identifiers in the reference dictionary the identification of the registration authority should be added to the identifier. The registration authority identification concept is defined in ISO/IEC 6523-1. At first, an organization identification scheme is selected from the registered schemes using the International Code Designator (ICD). Then, within this scheme, the organization is identified with the Organization Identifier (OI) (see Table 3).

For the IEC Common Data Dictionary, the ICD code "0112" shall be used standing for "Registration of Standardization Organization". For the Organization Identifier (OI), number "2" shall be used standing for "IEC".

A source identification for the data in IEC Common Data Dictionary is: "0112/2///61360_4". The added “61360_4” at the end is the Standards Identifier (SI) which is described in ISO 13584-26 as an extension to ISO/IEC 6523-1.

The IEC Common Data Dictionary may comprise multiple collections of dictionary items. Such collections do not create new identification spaces, i.e. identifiers shall be unique within the whole address space.

EXAMPLE 1 "0112/2///61987" for IEC 61987 series or "0112/2///62683" for IEC 62683.

EXAMPLE 2 The globally unique identification of a particular information object can be "0112/2///61340_4#AAF307#005".

Table 3 – Globally unique identification

Name	Description	Obligation	Data type	Maximum length
International Code Designator (ICD)	Identification of an organization identification scheme	Mandatory	Integer	4
Organization Identifier (OI)	Identification of an organization within an identification scheme	Mandatory	String	35

7.2 Specification of information object

Name: **Identifying_attributes**

Definition: collection of attributes identifying an information object

Description:

Figure 4 shows the detailed model of the information object "**Identifying_attributes**" using UML as presentation technique.

Identifying_attributes
code[1] version_number[1] revision_number[1] preferred_name[1] synonymous_name[0..2] short_name[0..1]

IEC

Figure 4 – Identifying attributes (UML class diagram)

7.3 Code

Attribute name: **code**

Attribute definition: unique six-character identifier of an information object

NOTE IEC 61360-2:2012 specifies 14 characters as the maximum length of the attribute “code”. The difference is caused by information appended to the code such as the International Registration Data Identifier (see ISO/IEC 11179-6:2005).

Description:

The first three characters shall be alphabetic, the last three numeric (format AAANNN). The character "X"[1] shall not be used as first character. The codes are issued sequentially and shall not have any relationship with the meaning of the information object.

In case of a modification of at least one attribute of the information object which affects one or both of the meaning and communication of the information object, a new (other) information object, having a new identifier, shall be defined.

Whether the identifier should be changed, so that a new information object is generated, or whether the version number of the information object should be changed, shall be determined for each case separately.

Such attributes are:

definition[2]**;**

unit_of_measure;

depends_on;

data_type.

Removing the relation to a **Condition_property**, i.e. changing the content of **depends_on**, shall result in a change of the **code**.

---

1 For local use within private environments, codes starting with XAA up to and including codes starting with XZZ may be used and are therefore excluded from the IEC Common Data Dictionary.

2 Extending the semantic context of an information object which does not violate the originally defined meaning by including another **Item_class** in the definition, results in a version change.

Obligation:	mandatory
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

7.4 Preferred_name

Attribute name:	**preferred_name**
Attribute definition:	single-word or multi-word designation assigned to an item
Description:	The content of **preferred_name** shall be identical to the name as used in international standards, if available. It is advised to keep the length of the name limited within 35 characters although its length may be extended up to 255 characters. See Annex F for conventions for names.
Obligation:	mandatory
Character type of values:	See Clause 26.

7.5 Revision_number

Attribute name:	**revision_number**
Attribute definition:	number used for administrative control of an information object
Description:	The **revision_number** of an information object should consist of 3 digits. Although the common ISO/IEC EXPRESS information model allows more digits, for the dictionary to be registered into IEC CDD or to be compatible with IEC 61360 series only 3 digits should be used. For specific purposes, revision numbers with more digits may be used. Consecutive revision numbers starting from "000" shall be issued in ascending order. Per information object, unique by its identifier, only one **revision_number** is current at any moment. A new revision of an information object shall be generated if an attribute of the information object is changed which neither affects its use nor its meaning, or when corrections of typing or spelling errors have been implemented. The **revision_number** shall be reset to its starting value "000" when the **version_number** is changed. See Annex D for rules for defining new versions or revisions of dictionary elements.
Obligation:	mandatory
Character type of values:	Digits "0" through "9".

7.6 Short_name

Attribute name: **short_name**

Attribute definition: shortened representation of the content of **preferred_name**

Description: The first character of the content of short_name shall be a letter:

– the short name should be preceded by a "@" character for a Condition_property;

– the short name shall be a short representation of the content in preferred_name. The length of the short name should be limited to 18 characters.

Obligation: optional

Character type of values: See Clause 26.

7.7 Synonymous_name

Attribute name: **synonymous_name**

Attribute definition: single-word or multi-word designation that differs from the given preferred name but represents the same concept

Description: The number of synonymous names shall be limited to two. For pragmatic reasons, the length of the **synonymous_name** should be limited to 70 characters.

Obligation: optional

Character type of values: See Clause 26.

7.8 Version_number

Attribute name: **version_number**

Attribute definition: number used to control the versions of an information object

Description: The **version_number** of an information object should consist of 3 digits. Although the common ISO/IEC EXPRESS information model allows more digits, for the information object to be registered into IEC CDD or to be compatible with IEC 61360 series only 3 digits should be used. For specific purposes, version numbers with more digits may be used. Consecutive version numbers starting from "001" shall be issued in ascending order. Per information object, unique by its identifier, only one version number is current at any time.

A new version of an information object shall be generated if at least one attribute of the information object has changed which can affect its use but which does not affect its meaning.

See Annex D for rules for defining new versions or revisions of dictionary elements.

Obligation: mandatory

Character type of values: Digits "0" through "9".

8 Semantic_attributes

8.1 Overview

The **Semantic_attributes** information object collects the attributes specifying the meaning of an information object.

8.2 Specification of information object

Name: **Semantic_attributes**

Definition: collection of attributes specifying the semantic content of an information object

Description:

Figure 5 shows the detailed model of the information object "**Semantic_attributes**" using UML as presentation technique.

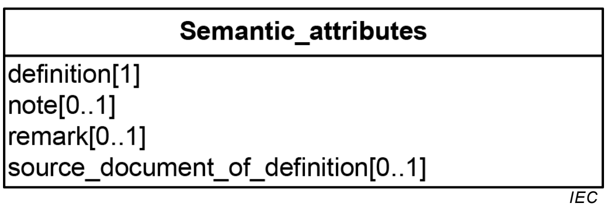

Figure 5 – Semantic attributes (UML class diagram)

8.3 Definition

Attribute name: **definition**

Attribute definition:	statement that describes the meaning of an information object in an unambiguous and unique manner to permit its differentiation from other information objects

EXAMPLE 1 Definition of "arcing distance":
arcing distance
value of the shortest distance in air external to the insulator between metallic parts normally having the operating voltage between them

Description: Conventions and requirements:

NOTE An information object standardized under former editions of IEC 61360-1 can be updated only by initiating a change request.

a) The content of **definition** of an information object shall be derived from the original definition as appearing in the latest corresponding IEC or ISO standards, if available.

b) Where possible, definitions of property information objects shall be independent from classes. Thus, reuse of properties as part of other class information objects is supported.

c) ISO 704 should be used as a basis for the writing of the definition statement.

d) The unit of measure shall not be included in the definition statement.

e) The level information should not occur in the definition statement.

   NOTE Level information can be specified with the data type **level_type**.

f) The semantic context should be included in the definition statement, if this is essential for the understanding of its meaning.

g) If the concept requires a limitation of its applicability, this shall be explicitly expressed in the definition statement.

   EXAMPLE 2 There exist different semantics of the term "rated voltage"; within products ≥ 1,5 kV, the term rated voltage expresses the maximum voltage for which a product is being designed and can be operated. This is currently not applicable to products less than 1,5 kV.

h) If dependency relations are an inherent part of the concept, these shall be included in the definition statement.

   EXAMPLE 3 Definition statement of the Property "reverse recovery time":

   "value of the time required for the reverse current of a diode to recover to a specified value, when switched from a specified forward current to a specified reverse voltage, at specified conditions"

i) In the case conditions are specified, the definition statement should end with the wording "at specified condition(s)".

j) If the concept represents an average value, the method of calculating the average shall be identified, by using a term that designates the method, such as “arithmetic mean”, “geometric mean”, “median”, or “mode”, either in the preferred_name or in the **definition**.

See Annex F for conventions for definitions.

Obligation: mandatory

Character type of values: See Clause 26.

8.4 Note

Attribute name: **note**

Attribute definition: statement providing further information on the **definition**, which is considered to be essential for its understanding

Description: EXAMPLE The **Property** "reverse recovery time" is further clarified by a **note**: "The reverse recovery time is measured as the time interval between t_0, the point where the forward current crosses the zero current axis, and the instant when for decreasing values of i_R a line through the points for 0,9 I_{RM} and 0,25 I_{RM} crosses the zero current axis."

Obligation: optional

Character type of values: See Clause 26.

8.5 Remark

Attribute name: **remark**

Attribute definition: additional information in text

Description: The content of **remark** shall not change the meaning of the **definition**.

Obligation: optional

Character type of values: See Clause 26.

8.6 Source_document_of_definition

Attribute name: **source_document_of_definition**

Attribute definition: reference to the source document from which the content of **definition** was derived

Description: The document shall be recognized by the ISO or IEC committee concerned as having wide acceptance and authoritative status as well as being publicly available.

Obligation: optional.

Character type of values: See Clause 26.

9 Administrative_attributes

9.1 Overview

The **Administrative_attributes** information object collects the attributes used to administer an information object.

9.2 Specification of information object

Name: **Administrative_attributes**

Definition: collection of attributes specifying the information used to manage an information object

Description:

Figure 6 shows the detailed model of the information object "Administrative_attributes" using UML as presentation technique.

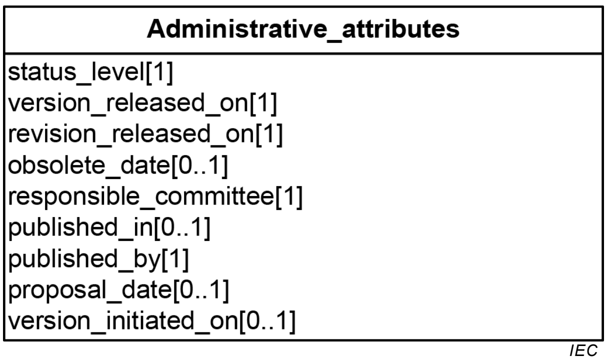

Figure 6 – Administrative_attributes (UML class diagram)

9.3 Obsolete_date

Attribute name: **obsolete_date**

Attribute definition: date when the status level (see 9.9) received the value "Superseded"

EXAMPLE "2010-06-28"

Obligation: conditional

Character type of values: date format according to "YYYY-MM-DD" as specified by ISO 8601

9.4 Proposal_date

Attribute name: **proposal_date**

Attribute definition: date when the information object was uploaded to the database

EXAMPLE "2003-05-14"

Obligation:	optional
Character type of values:	date format according to "YYYY-MM-DD" as specified by ISO 8601

9.5 Published_in

Attribute name:	**published_in**
Attribute definition:	identity number of the publicly available document in which the item and its definition were first published One or both of edition number and year should be included as appropriate. EXAMPLE "IEC 61175:2009"
Obligation:	optional
Character type of values:	See Clause 26.

9.6 Published_by

Attribute name:	**published_by**
Attribute definition:	organization responsible for the publication as defined by “published in” EXAMPLE "IEC"
Obligation:	optional
Character type of values:	See Clause 26.

9.7 Responsible_committee

Attribute name:	**responsible_committee**
Attribute definition:	Technical Committee or Subcommittee of IEC or ISO in charge of maintaining the information object EXAMPLE "SC3D"
Obligation:	mandatory
Character type of values:	See Clause 26.

9.8 Revision_released_on

Attribute name: **revision_released_on**

Attribute definition: date when the information object with the current revision number was authorized

EXAMPLE "2004-05-20"

Obligation: mandatory

Character type of values: date format according to "YYYY-MM-DD" as specified by ISO 8601

9.9 Status_level

Attribute name: **status_level**

Attribute definition: name of the stage of the standardization workflow the information object is in

Description: The following status levels may be assigned:

Proposal: status level of a new information object from its registration and identification in the database, until it has been accepted for work and a resolution has been reached on how to proceed following the preliminaries

Draft: status level of a new information object that has been accepted for work following the preliminaries with either the normal or extended database procedure, until the moment a decision has been taken on whether or not it is to be part of the standard

NOTE 1 The transition to **status_level** "draft" is from **status_level** "proposal".

Standard: status level of a new information object that has been released for use as part of the standard

NOTE 2 The transition to **status_level** "standard" is from **status_level** "draft".

Superseded: status level of an information object that is no longer part of the standard, irrespective of reason

NOTE 3 The transition to **status_level** "superseded" is from **status_level** "standard". On the information object page a note or a reference to a replacing information object further indicates the reason for obsolescence.

Rejected	status of an information object that has been entered into the database as part of a change request, but has not been approved to be part of the standard NOTE 4 The transition to "rejected" is either from **status_level** "proposal" or from **status_level** "draft".

Obligation: mandatory

Character type of values: See Clause 26.

9.10 Version_initiated_on

Attribute name: **version_initiated_on**

Attribute definition: date when the information object with the current version number was initially issued

EXAMPLE "2004-05-15"

Obligation: optional

Character type of values: date format according to "YYYY-MM-DD" as specified by ISO 8601

9.11 Version_released_on

Attribute name: **version_released_on**

Attribute definition: date when the information object with the current version number was authorized

EXAMPLE "2004-05-15"

Obligation: mandatory

Character type of values: date format according to "YYYY-MM-DD" as specified by ISO 8601

10 Value_attributes

10.1 Overview

The **Value_attributes** information object collects the attributes that specify the numerical or non-numerical value of a **Property**.

Every **Property** information object may have a value domain that covers the range of permissible values being either implicitly specified by the **value_format** or explicitly specified by listing the allowed values.

For quantitative **Property** information objects, the value domain may be expressed either as a value range providing the minimum and maximum values or as an explicit enumeration of discrete possible values.

For non-quantitative **Property** information objects, the value domain can be expressed by enumerating the permissible values in the form of text strings.

NOTE The attribute **value_format** (see 10.8) can be used as an indicator for the intended values that can be assigned to a **Property** during its usage. Additionally, it can be used for performing a basic quality control on the data provided.

For **Property** information objects with predefined value sets, a mechanism using values, represented by value codes and associated value meanings, is described in Clause 16.

If a **Property** serves as classifying property, a value list shall be specified.

10.2 Specification of information object

Name:	**Value_attributes**
Definition:	collection of attributes specifying the value(s) and unit(s) of an information object
Description:	

Figure 7 shows the detailed model of the information object "Value_attributes" using UML as presentation technique.

Value_attributes
data_type[1] value_format[0..1] unit_of_measure[0..1] alternative_units_of_measure[0..*] referenced_class_identifier[0..1] number_of_significant_digits[0..1]

IEC

Figure 7 – Value_attributes (UML class diagram)

10.3 Alternative_units_of_measure

Attribute name:	**alternative_units_of_measure**
Attribute definition:	units which may be used as an alternative to specify the value of a property NOTE 1 Alternative units allow the recommendation of further units in addition to the preferred unit (see 10.7). Reasons to choose an alternative unit are causes such as specific measurement principles, particular application domains, or special legislative systems. NOTE 2 Alternative units can be made available in purpose-related groups of units (see 24.8.3).

Description: Conventions and requirements:

a) Alternative units may be supplied in addition to a preferred unit. Such alternative units can, but do not have to be, SI units.

EXAMPLE bar, psi, or Torr instead of Pa.

b) Alternative units may be specified by using the code of the unit as specified in IEC TS 62720 or compliant unit dictionaries.

NOTE 3 The unit dictionaries can be supplied in computer-sensible form or as paper.

c) The conversion to a different unit is outside the scope of this document.

d) It is out of scope of this document to prescribe the use of a specific unit for transaction purposes including possible prefixes.

NOTE 4 **Property** information objects with **data_type** "CLASS_REFERENCE_TYPE" do not carry a unit.

e) It is strongly recommended not to specify the same concept in different properties if only the prefixes of the related units are different. Exceptions may occur if certain units are very common in particular business domains, such as "nm" (nanometre) in semiconductor manufacturing or "kV" (kilovolt) in the power distribution domain.

f) Each unit contained in the alternative_units_of_measure attribute is required to be associated with a string representation, whose text representation can be used for characterizing the unit at the instance level.

Obligation: optional

Character type of values: See Clause 26.

10.4 Data_type

10.4.1 General

Attribute name: **data_type**

Attribute definition: identifies a specific quality of the value of a **Property**

Description: The content of **data_type** is divided into categories:

– simple type;
– enumeration type;
– aggregate type;
– class instance type;
– class reference type;
– large object data;
– placement type;
– level type.

Obligation: mandatory

Character type of value: See Clause 26.

10.4.2 Simple type

A data type specifying that the value of a **Property** is a single numeric or a non-numeric value. The value format shall be defined according to 10.8.

In each designation of type where "_TYPE" is found, "_TYPE" may be omitted.

For a simple type, the valid data types are:

- BOOLEAN_TYPE

  The data type BOOLEAN_TYPE may be used for values representing truth values of logic or Boolean algebra or pattern of truth values.

  The use of "TRUE" as Boolean value indicates a true or accurate result. "FALSE" as Boolean value expresses a wrong or inaccurate result. Other values are deprecated.

  EXAMPLE 1 Deprecated Boolean values are values such as "YES" or "NO" and "Y" or "N".

  If a value contains a single truth value the data format shall contain "B1".

  NOTE 1 The use of BOOLEAN_TYPE values can lead to deficient specifications if the meaning of the values is not properly specified. In many cases the use of a value list instead of BOOLEAN_TYPE values provides more precise information.

  Additionally, some types of presentations can become meaningless if BOOLEAN_TYPE values are used.

  EXAMPLE 2 In circuit diagrams, lead texts are often omitted because of space restrictions or avoidance of the need for translation, and only the values are presented. Figure 8 shows examples for technical data associated with connecting lines presented in an abbreviated form (see IEC 61082:2014, Figure 38). In such cases, the use of values of BOOLEAN_TYPE could lead to presentations that are difficult to interpret, as such values are meaningless without lead text.

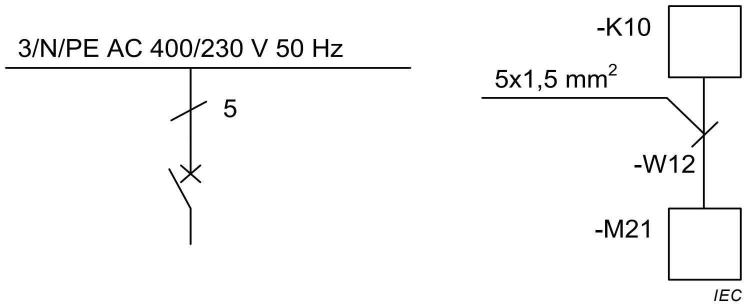

Figure 8 – Examples for technical data associated with connecting lines

- BINARY_TYPE

  The data type BINARY_TYPE may be used for values containing a sequence of bits, each bit being represented by "0" or "1". The value format shall be "B*n*" with n > 1. The textual part of the property, i.e. content of the attributes "definition", "note" or "remark", should give guidance about the interpretation of the bit pattern.

  NOTE 1 There is no upper limit specified for the value of "*n*". Current implementations (in 2014) support sequences of bits ("bitstrings") of a size up to 2 gigabytes.

- STRING_TYPE

  The data type STRING_TYPE may be used for values containing any sequence of elements, typically characters, using some character encoding.

Instead of the STRING_TYPE one of the more specific data types may be used:

- TRANSLATABLE_STRING_TYPE;
- NON_TRANSLATABLE_STRING_TYPE;
- NON_QUANTITATIVE_CODE_TYPE (deprecated);
- DATE_TIME_TYPE;
- DATE_TYPE;
- TIME_TYPE;
- IRDI_STRING;
- URI_TYPE.
- HTML5_TYPE.

• TRANSLATABLE_STRING_TYPE

The data type TRANSLATABLE_STRING_TYPE may be used for values containing strings, but that are supposed to be represented as different strings in different languages.

• NON_TRANSLATABLE_STRING_TYPE

The data type NON_TRANSLATABLE_STRING_TYPE may be used for values containing strings but that shall be represented in the same way in any language, i.e. such strings shall not be translated.

EXAMPLE 3 The MAC address of a computer, e.g. "00-80-41-ae-fd-7e".

• DATE_TYPE

The data type DATE_TYPE may be used for values containing a calendar date specified according to a representation compliant with ISO 8601.

A value whose data type is DATE_TYPE shall comply with the following lexical representation, which is a subset of the lexical representations allowed by ISO 8601. The lexical representation for DATE_TYPE is the reduced (right truncated) lexical representation for DATE_TIME_TYPE: "YYYY-MM-DD". No left truncation is allowed. An optional following time zone qualifier is allowed as for DATE_TIME_TYPE. To accommodate year values outside the range from "0001" to "9999", additional digits may be added to the left of this representation and a preceding "-" sign is allowed.

EXAMPLE 4 "1999-05-31" is the DATE_TYPE representation of: "31 May 1999".

Attribute **format** should contain the value "M..10".

• TIME_TYPE

The data type TIME_TYPE may be used for values defining a time specified according to a representation compliant with ISO 8601. A value of TIME_TYPE represents an instant of time that recurs every day.

NOTE 2 Only a subset of the lexical representations allowed by ISO 8601 is allowed for values of TIME_TYPE.

NOTE 3 The above restriction of ISO 8601 representations is the same as the one defined by XML Schema.

NOTE 4 Since the lexical representation allows an optional time zone indicator, TIME_TYPE values are partially ordered because it is not possible to determine the order of two values one of which has a time zone and the other does not.

A value whose data type is TIME_TYPE shall comply with the following lexical representation, which is a subset of the lexical representations allowed by ISO 8601. The lexical representation for TIME_TYPE is the left truncated lexical representation for DATE_TIME_TYPE "hh:mm:ss.sss" with optional following time zone indicator.

EXAMPLE 5 "13:20:00-05:00" is the TIME_TYPE representation of: 1.20 p.m. for Eastern Standard Time, which is 5 hours behind Coordinated Universal Time (UTC).

Attribute **format** should contain the value "M..14".

• DATE_TIME_TYPE

The data type DATE_TIME_TYPE may be used for values defining an instant of time specified according to a particular representation compliant with ISO 8601.

A value whose data type is DATE_TIME_TYPE shall comply with the following lexical representation, which is a subset of the lexical representations allowed by ISO 8601. This lexical representation is the ISO 8601 extended format "YYYY-MM-DDThh:mm:ss" where "YYYY" represents the year, "MM" the month and "DD" the day, preceded by an optional leading "-" sign. The letter "T" is the date/time separator and "hh", "mm", "ss" represent hour, minute and second, respectively. Additional digits can be used to increase the precision of fractional seconds if desired, i.e. the format "ss.ss..." with any number of digits after the decimal point is supported. The fractional second part is optional; other parts of the lexical form are not optional. To accommodate year values greater than "9999" additional digits can be added to the left of this representation. Leading zeros are required if the year value would otherwise have fewer than four digits; otherwise they are forbidden. The year "0000" is prohibited. The "YYYY" field shall have at least four digits, the "MM", "DD", "SS", "hh", "mm" and "ss" fields exactly two digits each (not counting fractional seconds); leading zeroes shall be used if the field would otherwise have too few digits. This representation may be immediately followed by a "Z" to indicate "Coordinated Universal Time (UTC)" or, to indicate the time zone, i.e. the difference between the local time and Coordinated Universal Time, immediately followed by a sign, "+" or "-", followed by the difference from UTC represented as "hh:mm". If the time zone is included, both hours and minutes shall be present.

ISO 8601 provides details about legal values in the various fields. If the time zone is included, both hours and minutes shall be present.

EXAMPLE 6 To indicate 1:20 p.m. on 31 May 1999 for Eastern Standard Time which is 5 hours behind Coordinated Universal Time (UTC), one would write: "1999-05-31T13:20:00-05:00".

NOTE 5 The above restriction of ISO 8601 representations is the same as the one defined by XML Schema.

Attribute **format** should contain the value "M..25".

- IRDI_STRING_TYPE

  The data type IRDI_STRING_TYPE may be used for values conforming to ISO/IEC 11179 series global identifier sequences.

  IRDI, DI and **code** have the following relationship:

  IRDI::= RAI'#'DI'#'VI

  NOTE 6 RAI stands for the registration authority identifier, DI for the data identifier, and VI for the version identifier. **Code** attribute is used as DI in this document.

  NOTE 7 In implementation, a data validation can be applied to check whether an information object exists for the referenced IRDI code.

  IRDI_STRING_TYPE may be used instead of one of the more specific data types:

  – ICID_STRING_TYPE;

  – ISO_29002_IRDI.

- ICID_STRING_TYPE

  The data type ICID_STRING_TYPE may be used for values conforming to IRDI_STRING_TYPE, where the delimiter between RAI and DI is “#” while the delimiter between DI and VI is confined to “##”.

  ICID, DI and **code** have the following relationship:

  ICID::= RAI'#'DI'##'VI

  NOTE 8 RAI stands for the registration authority identifier, DI for the data identifier, and VI for the version identifier. **Code** attribute is used as DI in this document.

  NOTE 9 In implementation, a data validation can be applied to check whether an information object exists for the referenced ICID code.

- ISO_29002_IRDI_TYPE

The data type ISO_29002_IRDI_TYPE may be used for values containing a global identifier that identifies an administrated item in a registry. The structure of this identifier complies with identifier syntax defined in ISO/TS 29002-5. The identifier shall fulfill the requirements specified in ISO/TS 29002-5 for an "international registration data identifier" (IRDI).

IRDI, DI and **code** have the following relationship:

IRDI::= RAI'#'DI'#'VI

NOTE 10 RAI stands for the registration authority identifier, DI for the data identifier, and VI for the version identifier. **Code** attribute is used as DI in this document.

As specified in ISO/TS 29002-5, the length of the identifier string shall not be greater than 290 characters.

NOTE 11 According to ISO/TS 29002-5 an IRDI consists either of a string that does not contain the ''#' character, to identify an organization, or of three sub-strings that do not contain the '#' character and that are separated by the '#' character to identify any other administrated item.

NOTE 12 If IRDI is not used for identifying an organization:

– the first sub-string, called the "Registration Authority Identifier (RAI)", identifies the organization which is responsible for the administration of the administrated item;
– the second sub-string, called the "Data Identifier (DI)", contains both a categorization of the administrated item, represented by two characters followed by the minus ("-") sign as defined in ISO/TS 29002-5 (e.g. "class", "property", "unit"), and the identifier assigned to the administrated item by the RAI;
– the third sub-string corresponds to the "Version Identifier (VI)" of the IRDI.

- URI_TYPE

The data type URI_TYPE may be used for values containing any sequence of characters that is a URI.

NOTE 13 A URI_type allows in particular to express a URL.

- HTML5_TYPE

The data type HTML5_TYPE may be used for values containing strings with any sequence of characters, using the syntax of HTML5 (see W3C Recommendation 28:2014). This content provides for a pictorial presentation that can be impossible to achieve by solely using characters.

NOTE 14 As HTML5 supports MathML (see W3C Recommendation 10:2014), mathematical formulas can be presented.

EXAMPLE 7 The value $\frac{1}{\sqrt{3}}420\,\mathrm{kV}$ (242,48711305964282109384248781082 kV) can be expressed with HTML5_TYPE as

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
  </head>
  <body>
    <math xmlns="http://www.w3.org/1998/Math/MathML">
      <mrow>
        <mrow>
          <mfrac>
            <mn>1</mn>
            <msqrt>
              <mn>3</mn>
            </msqrt>
          </mfrac>
        </mrow>
        <mo>·</mo>
        <mn>420</mn>
        <mi>kV</mi>
      </mrow>
    </math>
```

```
  </body>
</html>
```

Cases exist where, in addition to the presentation of the value, its exact numeric value is of interest too. In such cases reference to an **Item_class** is recommended that provides a **Property** for the numeric value (data type INTEGER, REAL, etc.) and a **Property** for the HTML presentation (data type HTML5).

NOTE 15 The class is referenced by using CLASS_INSTANCE_REFERENCE.

- NUMBER_TYPE

  The data type NUMBER_TYPE may be used for values containing numbers that can be written as numeric values. This data type may be used when a more specific numeric representation is not important or unwanted.

  Instead of the NUMBER_TYPE one of the more specific data types may be used:

  – INTEGER;

  – RATIONAL;

  – REAL.

- INTEGER, INT_TYPE

  The data type INT_TYPE may be used for values containing numbers that can be written as positive or negative whole numbers or as zero.

  Instead of the INT_TYPE one of the more specific data types may be used:

  – INT_MEASURE_TYPE;

  – INT_CURRENCY_TYPE.

- INT_MEASURE_TYPE

  The data type INT_MEASURE_TYPE may be used for values containing values that are measures of type INTEGER.

  In addition, the data type specifies a unit or a unit identifier, in which the values are expressed. It may also specify alternative units or alternative unit identifiers that are valid if the value is explicitly associated with one of these units.

  Either a unit or a unit identifier is mandatory. In cases where both are provided, the unit takes precedence.

  NOTE 16 The unit identifier used in **code_of_unit**, **code_of_alternative_unit**, or **code_of_list_of_units** attributes can resolve to a unit provided by a server for units of measure.

- INT_CURRENCY_TYPE

  The data type INT_CURRENCY_TYPE may be used for values containing values that are integer currencies.

- RATIONAL_TYPE

  The data type RATIONAL_TYPE may be used for values containing values that are of type rational.

  EXAMPLE 8 $^1/_2$, $^3/_4$ or $^7/_2$ are values that can be represented with RATIONAL_TYPE.

  Instead of the RATIONAL_TYPE the more specific data type

  – RATIONAL_MEASURE_TYPE

  may be used.

- RATIONAL_MEASURE_TYPE

  The data type RATIONAL_MEASURE_TYPE may be used for values containing values that are measures of type RATIONAL.

  EXAMPLE 9 A very common diameter for water pipes is "¾ inch".

In addition, the data type specifies a unit or a unit identifier, in which the values are expressed. It may also specify alternative units or alternative unit identifiers that are valid if the value is explicitly associated with one of these units.

Either a unit or a unit identifier is mandatory. In cases where both are provided, the unit takes precedence.

NOTE 17 The unit identifier used in **code_of_unit**, **code_of_alternative_unit**, or **code_of_list_of_units** attributes can resolve to a unit provided by a server for units of measure.

- REAL_TYPE

  The data type REAL_TYPE may be used for values containing numbers that can be written as a terminating or non-terminating decimal; a rational or irrational number.

  Instead of the REAL_TYPE, one of the more specific data types may be used:

  – REAL_MEASURE_TYPE;

  – REAL_CURRENCY_TYPE.

- REAL_MEASURE_TYPE

  The data type REAL_MEASURE_TYPE may be used for values containing values that are measures of type REAL.

  In addition, the data type specifies a unit or a unit identifier in which the values are expressed. It may also specify alternative units or alternative unit identifiers that are valid if the value is explicitly associated with one of these units.

  Either a unit or a unit identifier is mandatory. In cases where both are provided, the unit takes precedence.

  NOTE 18 The unit identifier used in **code_of_unit**, **code_of_alternative_unit**, or **code_of_list_of_units** attributes can resolve to a unit provided by a server for units of measure.

- REAL_CURRENCY_TYPE

  Data type containing real values of currencies.

10.4.3 Enumeration type

The enumeration types may be used for specifying that the content of a value shall be taken from a predefined set of values.

In each designation of type where "_TYPE" is found, "_TYPE" may be omitted.

For an enumeration type, the valid data types are:

- ENUM_CODE_TYPE(enum_id), ENUM_CODE_TYPE(enum_id(code1, code2, ...))

  The data type ENUM_CODE_TYPE may be used for values referencing a set of strings or numerals, as values, each representing assigned meanings.

  NOTE 1 enum_id is a global identifier for the enumeration list.

  NOTE 2 code1, code2, ... are the value codes of enumerated values.

- ENUM_INT_TYPE(enum_id), ENUM_INT_TYPE(enum_id(code1, code2, ...))

  The data type ENUM_INT_TYPE may be used for values referencing a set of integer values as values.

  NOTE 3 enum_id is a global identifier for the enumeration list.

  NOTE 4 code1, code2, ... are the value codes of enumerated values.

- ENUM_REAL_TYPE(enum_id), ENUM_REAL_TYPE(enum_id(code1, code2, ...))

  The data type ENUM_REAL_TYPE may be used for values referencing a set of real values as values.

  NOTE 5 enum_id is a global identifier for the enumeration list.

NOTE 6 code1, code2, ... are the value codes of enumerated values.

NOTE 7 "ENUM_REAL_TYPE" is not available in ISO 13584-42/ IEC 61360-2, instead a constraint mechanism named “enumeration constraint” can be used for the purpose. However, the specification of the enumeration constraint is much more complex than ENUM_REAL, because the former needs to define first the data type, and then constrain the selectable subset, with an implicit understanding that only one of them is selected.

- ENUM_STRING_TYPE(enum_id), ENUM_STRING_TYPE(enum_id(code1, code2, ...))

  The data type ENUM_STRING_TYPE may be used for values referencing a set of string values, as values, each representing assigned meanings.

  NOTE 8 "enum_id" is the global identifier of the enumeration list.

  NOTE 9 "code1", "code2", ... are the codes of the enumerated values.

- ENUM_RATIONAL_TYPE(enum_id), ENUM_RATIONAL_TYPE(enum_id(code1, code2, ...))

  The data type ENUM_ RATIONAL_TYPE may be used for values referencing a set of string values, as values, each representing a rational number.

  NOTE 10 "enum_id" is the global identifier of the enumeration list.

  NOTE 11 "code1", "code2", ... are the codes of the enumerated values.

- ENUM_REFERENCE_TYPE(enum_id), ENUM_INSTANCE_TYPE(enum_id), ENUM_REFERENCE_TYPE(enum_id(code1, code2, ...)), ENUM_INSTANCE_TYPE(enum_id(code1, code2, ...))

  The data type ENUM_REFERENCE_TYPE/ ENUM_INSTANCE_TYPE may be used for values referencing a set of string values, as values, each representing the global identifier of a class.

  NOTE 12 "enum_id" is the global identifier of the enumeration list.

  NOTE 13 "code1", "code2", ... are the codes of the enumerated values.

- ENUM_BOOLEAN_TYPE(enum_id), ENUM_BOOLEAN_TYPE((enum_id(code1, code2, ...))

  The data type ENUM_BOOLEAN_TYPE may be used for values referencing a set of Boolean values.

  NOTE 14 "enum_id" is the global identifier of the enumeration list.

  NOTE 15 "code1", "code2", ... are the codes of the enumerated values.

10.4.4 Class instance type

- CLASS_INSTANCE_TYPE(icid)

  The data type CLASS_INSTANCE_TYPE may be used for values containing a reference to an **Item_class**. The referenced **Item_class** may reside in any part of the classification hierarchy and contributes a collection of zero, one or more **Property** information objects. The value of the referencing **Property** shall contain the **code** of the referenced **Item_class**.

  A CLASS_INSTANCE_TYPE references an **Item_class** in a way similar to CLASS_REFERENCE_TYPE (see 10.4.5), but unlike the CLASS_REFERENCE_TYPE, instances conforming to the referenced **Item_class** shall be kept together with the referencing class, i.e. are internal to the instances of the referencing class. This means if there is an **Item_class** A with instances A_1, A_2, ..., A_n, where A_1 references instance B_1 of **Item_class** B, then the instances A_2, ..., A_n of A reference to different instances of B.

  In other words, CLASS_INSTANCE_TYPE assumes that each referencing instance A_1, A_2, ..., A_n of **Item_class** A is linked to its "personal" instance of B. Thus, if A_1 references B_1 and in the referenced **Item_class** instance B_1, an instance value is changed, the change is not visible from all (other) referencing **Item_class** instances A_2, ..., A_n.

NOTE 1 The "icid" in CLASS_REFERENCE_TYPE can omit "VI" (version identifier), in this case the class having the specified identifier with the latest version can be referenced.

NOTE 2 See IEC 62656-1 for detailed information on the structure of the global identifier ICID.

10.4.5 Class reference type

- CLASS_REFERENCE_TYPE(icid)

  The data type CLASS_ REFERENCE_TYPE may be used for values containing a reference to an **Item_class**. The referenced **Item_class** may reside in any part of the classification hierarchy and contributes a collection of zero, one or more **Property** information objects. The value of the referencing **Property** shall contain the **code** of the referenced **Item_class**.

  If there are classes A_1, A_2, ..., A_n that reference one and the same external **Item_class** information object B, then the instances of A_1, A_2, ..., A_n reference to the same instance B_1 of B. In other words, CLASS_REFERENCE_TYPE assumes that the instance stored at the side of the referenced **Item_class** B, is shared among all the referencing classes A_1, A_2, ..., A_n. Thus, if in the referenced **Item_class** an instance value is changed, the change is visible from all referencing **Item_class** information objects.

  NOTE 1 The "icid" in CLASS_REFERENCE_TYPE can omit "VI" (version identifier). In this case the reference to the latest version of the referenced class is made.

  NOTE 2 See IEC 62656-1 for detailed information on the structure of the global identifier ICID.

10.4.6 Aggregate type

Data type containing collections of two or more values. Individual values within the aggregate can be identified by a numeric index. One-dimensional sets of values may be represented as bag, list, or set. Multi-dimensional sets of values can be expressed as arrays.

The format applicable to each value of the aggregate shall be defined according to 10.4.2.

NOTE 1 An aggregate connects to a simple type by using the keyword "OF".

- BAG(*b1*, *b2*)

  A BAG data type has as its domain unordered collections of like constituents. The lower and upper bounds, which are integer-valued, define the minimum and maximum number of constituents that can be held in the collection defined by a BAG data type.

  Rules and restrictions:

  a) The *b1* expression shall be an integer value greater than or equal to zero. It gives the lower bound, which is the minimum number of constituents that can be in a bag. *b1* shall not be the indeterminate value ("?").

  b) The *b2* expression shall be an integer value greater than or equal to *b1*, or an indeterminate value ("?"). It gives the upper bound, which is the maximum number of constituents that can be in a bag. If this value is indeterminate ("?"), the number of constituents in a bag is not being bounded from above.

  c) If the boundary specification is omitted, the limits are (0,?).

- LIST(*b1*, *b2*)

  A LIST data type has as its domain sequences of like constituents. The lower and upper bounds, which are integer-valued, define the minimum and maximum number of constituents that can be held in the collection defined by a LIST data type.

  Rules and restrictions:

  a) The *b1* expression shall be an integer value greater than or equal to zero. It gives the lower bound, which is the minimum number of constituents that can be in a list. *b1* shall not be the indeterminate value ("?").

  b) The *b2* expression shall be an integer value greater than or equal to *b1*, or an indeterminate value ("?"). It gives the upper bound, which is the maximum number of constituents that can be in a list. If this value is indeterminate ("?"), the number of constituents in a list is not being bounded from above.

c) If the boundary specification is omitted, the limits are (0,?).

- UNIQUE_LIST(*b1*, *b2*)

  The value has the form of a LIST in which duplicate values shall not occur.

  Rules and restrictions:

  a) The rules and restrictions of the LIST data type apply.

  b) Each constituent in an occurrence of a UNIQUE_LIST data type shall be different from (that is, not instance equal to) every other constituent in the same list.

- SET(*b1*, *b2*)

  A SET data type has as its domain unordered collections of like constituents. The lower and upper bounds, which are integer-valued, define the minimum and maximum number of constituents that can be held in the collection defined by a SET data type.

  Rules and restrictions:

  a) The *b1* expression shall be an integer value greater than or equal to zero. It gives the lower bound, which is the minimum number of constituents that can be in a set. *b1* shall not be the indeterminate value ("?").

  b) The *b2* expression shall be an integer value greater than or equal to *b1*, or an indeterminate value ("?"). It gives the upper bound, which is the maximum number of constituents that can be in a set value of this data type. If this value is indeterminate ("?"), the number of constituents in a set is not being bounded from above.

  c) If the boundary specification is omitted, the limits are (0,?).

  d) Each constituent in an occurrence of a SET data type shall be different from (that is, not instance equal to) every other constituent in the same set.

- CONSTRAINED_SET(*b1*, *b2*, *cmn*, *cmx*)

  The CONSTRAINED_SET data type has as its domain unordered collections of constituents that may be expressed as SET of values of which subsets may be extracted. The sizes of allowed subsets are defined by their minimal and maximal values. If these sizes do not exist, any subset is allowed.

  Rules and restrictions:

  a) The rules and restrictions of the SET data type apply.

  b) The *cmn* shall be an integer value greater than or equal to zero. It gives the lower bound, which is the minimum number of constituents that may be extracted. *cmn* shall not be the indeterminate value ("?").

  c) The *cmx* shall be an integer value greater than or equal to *cmn*, or an indeterminate value ("?"). It gives the upper bound, which is the maximum number of constituents that may be extracted.

- ARRAY(*b1*, *b2*)

  An ARRAY data type has as its domain indexed fixed-size collections of like constituents. The lower and upper bounds, which are integer-valued numbers, define the range of index values, and thus the size of each array collection.

  Given that m is the lower bound and n is the upper bound, there are exactly $n - m + 1$ constituents in the array. These constituents are indexed by subscripts from m to n, inclusive. The bounds may be positive, negative or zero, but shall not be indeterminate ("?")

  Rules and restrictions:

  a) Both values in the bound specification, *b1* and *b2*, shall be integer values. Neither shall contain the indeterminate (?) value.

b) *b1* gives the lower bound of the array. This shall be the lowest index which is valid for an array value of this data type.

c) *b2* gives the upper bound of the array. This shall be the highest index which is valid for an array value of this data type.

d) *b1* shall be less than or equal to *b2*.

- OPTIONAL_ARRAY(*b1*, *b2*)

  The value has the form of an ARRAY in which indeterminate values ("?") may occur.

  Rules and restrictions:

  a) The rules and restrictions of the ARRAY data type apply.

  b) An array value of this data type may have the indeterminate value ("?") at one or more index positions.

- UNIQUE_ARRAY(*b1*, *b2*)

  The value has the form of an ARRAY in which duplicate values shall not occur.

  Rules and restrictions:

  a) The rules and restrictions of the ARRAY data type apply.

  b) Each constituent in the array shall be different from (that is, not instance equal to) every other constituent in the same array.

- UNIQUE_OPTIONAL_ARRAY(*b1*, *b2*)

  The value has the form of an ARRAY in which duplicate values shall not occur. Indeterminate values ("?") may occur.

  Rules and restrictions:

  a) The rules and restrictions of the ARRAY data type apply.

  b) An array value of this data type may have the indeterminate value ("?") at one or more index positions.

  c) Each constituent in an array shall be different from (that is, not instance equal to) every other constituent in the same array value.

10.4.7 Level type

- LEVEL_TYPE(MIN,NOM,TYP,MAX)

  The data type LEVEL_TYPE may be used for values representing up to four variants of a numeric value which may be given as

  – INT_TYPE;

  – INT_MEASURE_TYPE;

  – REAL_TYPE;

  – REAL_MEASURE_TYPE.

  Each variant of this value expresses the number value in a specific role:

  – MIN minimum of the value;

  – NOM value as designated;

  – TYP value as typically present;

  – MAX maximum of the value.

  As the interpretation of the level type values relies on the fixed sequence of the levels, all levels specified for a property have to be represented. There is no obligation to fill all levels with values although it is advised to do so for data consistency reasons.

  EXAMPLE 1 In the case of having a property which is of the LEVEL_TYPE min/nom/typ/max and

  – only the "MIN" value shall be transmitted, each of the other values should contain a "NIL" value;

  – only the "MIN" and "MAX" values shall be transmitted, each of the other values should contain a "NIL" value.

EXAMPLE 2 In the case of having a property which is of the LEVEL_TYPE min/max – expressing a range – only those two values need be provided.

NOTE The separator sign used to separate the values of the defined levels depends on the implementation.

The use of the level type should be restricted to those **Property** information objects that are applicable in domains where the reporting of multiple values on a single characteristic is recognized as common practice.

EXAMPLE 3 Values used in industries such as the electronic component industry.

Limiting values can be expressed by defining a **Property** specifying the levels "MIN" or "MAX" or both.

EXAMPLE 4 A 12 V (nominal) car battery has 6 cells with a typical voltage of about 2,2 V each, giving a typical battery voltage of about 13,5 V. On charge, the voltage can reach a maximum of about 14,5 V but it is considered fully discharged when the voltage falls below a minimum of 12,5 V.

A specific use of LEVEL_TYPE is the concept of limiting values. When the qualification “limiting” is added to a **Property**, which may be expressed in its **definition**, the property (together with its value) specifies a threshold beyond which a device incurs damage resulting in permanent unwanted changes of functional or physical characteristics influencing the performance of the device.

EXAMPLE 5 In the case of the car battery (see EXAMPLE 4), the limiting maximum value might be specified at a value of 16,5 V.

In the context of the limiting values, the levels "NOM" and "TYP" do not apply.

Aggregates may contain LEVEL_TYPE values.

EXAMPLE 6 LIST(1,?) OF LEVEL(MIN,NOM,TYP,MAX) OF REAL_MEASURE_TYPE

10.4.8 Large object type

- LOB(*location*, *media_type*, *media_subtype*, *content_transfer_encoding*)

  A large object (LOB) is an information object that is logically stored as a value of a **Property** but may be physically stored independent (mostly separated) of the **Property** information object. Large objects typically store large amounts of data.

  EXAMPLE Video data, audio data, images, xml data, archives.

  A value of type LOB consists of a list of four values:

  – *location*: Specification of the place where the value of the large object may be accessed. This storage location should be specified in a way that all parties, i.e. the writer and the reader(s) of the LOB, can access the data.

  – *media_type*: Specification of the type of the LOB value specified in accordance with the multipurpose internet mail extensions (MIME), Part 1. See IETF RFC 2045:1996.

  – *media_subtype*: Specification of the subtype of the LOB value specified in accordance with the multipurpose internet mail extensions (MIME), Part 2. See IETF RFC 2046:1996.

  – *content_transfer_encoding*: Specification of the character encoding of the LOB value specified in accordance with the multipurpose internet mail extensions (MIME), Part 3. See IETF RFC 2047:1996.

10.4.9 Placement type

Data type containing direction information for locating an item with respect to the coordinate system of its geometric context. Placement type values provide the information for setting up the local coordinate system required for locating the item.

NOTE 1 A detailed specification of the mathematical methods used by the placement types can be found in IEC 62656-1 and ISO 10303-42.

NOTE 2 In the case of ISO 10303-42, AXIS1_PLACEMENT, AXIS2_PLACEMENT_2D, AXIS2_PLACEMENT_3D inherit the attribute “location” that is of CARTESIAN_POINT type in 2D or 3D space, from the generic geometric entity “placement”. In the case of IEC 61360-1, this is achieved by associating a condition type **Property** of the placement type, i.e. CARTESIAN_POINT, to the geometric shape that uses either one of AXIS1_PLACEMENT_2D, AXIS1_PLACEMENT_3D, AXIS2_PLACEMENT_2D, or AXIS2_PLACEMENT_3D data types.

- AXIS1_PLACEMENT_2D(ref_dir_X, ref_dir_Y)

  The values specify a reference direction in two-dimensional space for use as line of symmetry for bodies that are line symmetric.

  – ref_dir_X: Component of the direction vector of the reference direction in X direction of the geometric context coordinate system.

  – ref_dir_Y: Component of the direction vector of the reference direction in Y direction of the geometric context coordinate system.

- AXIS1_PLACEMENT_3D(ref_dir_X, ref_dir_Y, ref_dir_Z)

  The values specify a reference direction in three-dimensional space for use as line of symmetry for bodies that are line symmetric.

  – ref_dir_X: Component of the direction vector of the reference direction in X direction of the geometric context coordinate system.

  – ref_dir_Y: Component of the direction vector of the reference direction in Y direction of the geometric context coordinate system.

  – ref_dir_Z: Component of the direction vector of the reference direction in Z direction of the geometric context coordinate system.

- AXIS2_PLACEMENT_2D(ref_dir_X, ref_dir_Y)

  The values specify the local X axis of the local coordinate system used for locating an item.

  – ref_dir_X: Component of the direction vector of the local X axis in X direction of the geometric context coordinate system.

  – ref_dir_Y: Component of the direction vector of the local X axis in Y direction of the geometric context coordinate system.

- AXIS2_PLACEMENT_3D(($ref1\_dir_X$, $ref1\_dir_Y$, $ref1\_dir_Z$), ($ref2\_dir_X$, $ref2\_dir_Y$, $ref2\_dir_Z$))

  The values specify the local Z axis and the direction used to determine the direction of the X axis of the local coordinate system used for locating an item.

  – $ref1\_dir_X$: Component of the direction vector of the local Z axis in X direction of the geometric context coordinate system.

  – $ref1\_dir_Y$: Component of the direction vector of the local Z axis in Y direction of the geometric context coordinate system.

  – $ref1\_dir_Z$: Component of the direction vector of the local Z axis in Z direction of the geometric context coordinate system.

  – $ref2\_dir_X$: Component of the direction vector of the direction used to determine the direction of the local X axis in X direction of the geometric context coordinate system.

  – $ref2\_dir_Y$: Component of the direction vector of the direction used to determine the direction of the local X axis in Y direction of the geometric context coordinate system.

  – $ref2\_dir_Z$: Component of the direction vector of the direction used to determine the direction of the local X axis in Z direction of the geometric context coordinate system.

10.4.10 Data type dependencies

As specified in 10.1, **Property** information objects can have numerical or non-numerical values according to the specifications in the attributes **data_type** and **value_format**. Nested values such as aggregates containing other aggregates may occur.

Table 4 gives an overview of the allowed combinations of **data_type** values, (see 10.4.2 to 10.4.9):

– an allowed combination is marked with: “ok”;

– a combination that is not allowed is marked with a minus sign: "–".

Possible content of **value_format** (see 10.8) is listed, too.

EXAMPLE 1 A **Property** can have **data_type** SET (category: Aggregate type) of REAL_MEASURE_TYPE (category: Real), which is marked with "ok" in Table 4.

EXAMPLE 2 A **Property** with **data_type** LIST (category Aggregate type) of ENUM_INT_TYPE (category Enumeration type) is deprecated and marked with "–" in Table 4.

When specifying a new **Property**, the following sequential steps can be taken to define the complete data type by process of elimination in the given sequence:

– choose property;
– choose data type;
– choose data value range;
– choose value format.

Table 4 – Data type dependencies

	Category of data_type (see 10.4.1)								
	Simple type	**Aggregate type**	**Enumeration**	**Class instance type**	**Class reference type**	**Large object data**	**Placement type**	**Level type**	
Category of content:									**Content of attribute value_format (see 10.8):**
Simple type	–	ok	–	–	–	–	–	[d]	A, M, N, X, B, NR1, NR2, NR3, NR4, HTML5
Aggregate type	–	ok	–	–	–	[a]	–	–	
Enumeration type	–	–	–	–	–	–	–	–	
Class instance type	–	[b]	–	–	–	–	–	–	
Class reference type	–	[b]	–	–	–	–	–	–	
Large object data	–	–	–	–	–	–	–	–	
Placement type	–	ok	–	–	–	–	–	–	
Level type	ok	–	–	–	–	–	–	–	
Boolean/ Binary	ok	ok	–		–	–	–	–	B
Integer	ok	ok	ok	–	–	–	–	ok	NR1
Real	ok	ok	–	–	–	–	–	ok	NR2, NR3
Rational	ok	ok	–	–	–	–	–	ok	NR4
String	ok	ok	[c]	[b]	[b]	–	ok	–	A, M, N, X, HTML5

[a] Array containing the pointer to the LOB information object.

[b] Pointer to **Item_class** information object.

[c] Pointer to **Value_list** information object.

[d] Only numeric values of type number.

10.5 Number_of_significant_digits

Attribute name: **number_of_significant_digits**

Attribute definition: prescription of the precision that shall be obtained when assigning a numerical value to a property

Description: Conventions and requirements:

a) Trailing zeros without a decimal mark indicate lack of significant digits unless a decimal mark is appended.

EXAMPLE 1 "1000" has one significant digit and "1000," has four significant digits.

b) All digits (following a non-zero digit) are significant once a decimal mark is used, so trailing zeros located to the right of the decimal mark are to be considered significant. Zeros with significant digits on each side of the decimal mark are also significant.

EXAMPLE 2 "30,4", "34,0", and "3,40" each have three significant digits but "340" has only two significant digits.

c) Leading zeros are not treated as significant.

EXAMPLE 3 One or more leading zeroes are not treated as significant, e.g. 00257,7 has 4 significant digits, and 0,00492 has 6 significant digits.

d) The sign of the number is independent and is not counted within the significant digits.

EXAMPLE 4 +30,4", "-34,0", and "3,40" each have three significant digits.

Obligation: optional

Character type of values: See Clause 26.

10.6 Referenced_class_identifier

Attribute name: **referenced_class_identifier**

Attribute definition: link to an **Item_class** information object

Description: The class referenced contains a set of **Property** information objects. The **Property** information object from which the reference is made shall have the **value_format** "CLASS_INSTANCE_TYPE" or "CLASS_REFERENCE_TYPE" (see 10.4.4 and 10.4.5).

Obligation: optional

Character type of values: Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

10.7 Unit_of_measure

Attribute name: **unit_of_measure**

Attribute definition: prescription of the unit in which the value of a property should be expressed

Description:	Conventions and requirements: a) Preference shall be given to SI units without decimal multiplier prefixes as defined in the IEC 80000 and ISO 80000 series. b) Alternative units may be specified by using the code of the unit as specified in IEC TS 62720 or compliant unit dictionaries. NOTE 1 The unit dictionaries can be supplied in computer-sensible form or as paper. c) The conversion to a different unit is outside the scope of this document. d) It is out of scope of this document to prescribe the use of a specific unit for transaction purposes including possible prefixes. e) It is strongly recommended not to specify the same concept with different **Property** information objects if only the prefixes of the related units are different. Exceptions may occur if certain units are common in particular business domains, such as "nm" (nanometre) in semiconductor manufacturing or "kV" (kilovolt) in power distribution area. f) Each unit contained in the **unit_of_measure** attribute is required to be associated with a string representation, whose text representation can be used for characterizing the unit at the instance level. NOTE 2 **Property** information objects with **data_type** "CLASS_REFERENCE_TYPE" do not carry a unit. EXAMPLE The IRDI of unit "%" is "0112/2///62720#UAA000#01"[3].
Obligation:	conditional
Condition:	**unit_of_measure** shall be specified for quantitative **Property** information objects
Character type of values:	See Clause 26.

10.8 Value_format

Attribute name:	**value_format**
Attribute definition:	specification of type and length of the representation of the value of a **Property**
Obligation:	optional
Character type of values:	See Clause 26.

[3] Use of the identifiers of the units is not restricted to IEC CDD but may be used outside IEC CDD.

Comments

The **value_format** is specified as part of a **Property** and can be used at implementation level for purposes of consistency and error checking in electronic exchange.

At implementation level, e.g. in the set-up of product information databases or electronic exchange, it can be treated as a default format that can be used as-is, re-defined (restricted or extended), or ignored entirely depending on the requirements and context in which the information is used.

For **Property** information objects of which the **data_type** has the value "class_reference_type", no **value_format** is requested.

See Annex G for the specification of syntax and content of **value_format**.

Below an overview of the content of **value_format** is provided:

a) Non-quantitative **value_format** types:

- A = alphabetic, letters only;
- M = mixed, all characters allowed;
- N = numeric, digits only;
- X = alphanumeric;
- B = binary, 0 or 1.

b) Quantitative **value_format** types:

- NR1 = integers;
- NR2 = rational numbers with decimal-mark (real numbers);
- NR3 = rational numbers with decimal-mark and exponent-mark (floating point numbers);
- S = signed (positive or negative numbers);
- **.** = decimal-mark;
- E = exponent-mark, base 10: (A)E(B) represents the value $A \times 10^{B}$.

c) Field length

The field length of a non-quantitative value shall be indicated by a number (e.g. 17). A variable field length shall start with two dots. The following preferred formats have been defined:

A..3	N..3	X..3	M..3	B 1
A..8	N..8	X..8	M..8	
A..17	N..17	X..17	M..17	
A..35	N..35 X..35	M..35		
A..($n \times$ 35)	N..($n \times$ 35)	X..($n \times$ 35)	M..($n \times$ 35)	

A fixed field length starts with one space. In such formats, no special characters shall occur.

EXAMPLE 1 "A 3", "N 8", "X 17", "M 35"

The field length of a quantitative value shall be indicated by a combination of digits and characters (e.g. "3.3ES2"). The following preferred standard formats have been defined:

NR1..4

NR1 S..4

NR2..3.3

NR2 S..3.3

NR3..3.3ES2

NR3 S..3.3ES2

A fixed field length starts with one space. In such formats no special characters shall occur.

EXAMPLE 2 "NR1 4", "NR1 S 4"

11 Condition_property

11.1 Specification of information object

Name:	**Condition_property**
Definition:	property information object that affects the value of another property.
Description:	Any instance of **Condition_property** shall contain the value "CONDITION_DET" in the (inherited) attribute **product_data_element_type**. Many properties have values which depend on the value(s) of one or more additional **Property** information objects. When the value of a condition is given as a range, it shall be specified by two **Condition_property** information objects, representing the upper and lower bound of this range.

NOTE **Condition_property** is a subtype of **Property**. Thus it inherits all relationships from the attribute classes of the supertype, i.e. identifying attributes, semantic attributes, administrative attributes, value attributes, and translation data.

Figure 9 shows the detailed model of the information object "**Condition_property**" using UML as presentation technique.

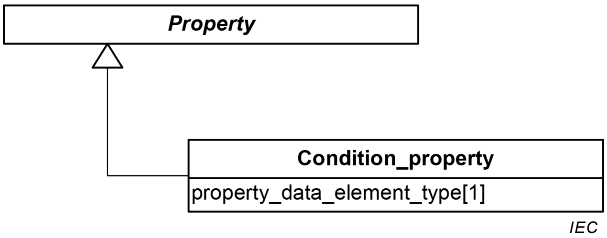

Figure 9 – Condition_property (UML class diagram)

11.2 Property_data_element_type

Attribute name:	**property_data_element_type**
Attribute definition:	role of the **Property**
Description:	One of the following roles shall be assigned:
	CONDITION_DET — **Property** acting as condition
Obligation:	mandatory
Character type of values:	See Clause 26.

12 Dependent_condition_property

12.1 Specification of information object

Name:	**Dependent_condition_property**
Definition:	condition that further depends on one or more other conditions
Description:	Any instance of **Dependent_condition_property** shall contain the value "DEPENDENT_C_DET" in the (inherited) attribute **product_data_element_type**. Many **Condition_property** information objects have values which again depend on the value(s) of one or more additional properties. When the value of a **Dependent_condition_property** information object is given as a range, it shall be specified by two **Dependent_condition_property** information objects, representing the upper and lower bound of this range.

NOTE **Dependent_condition_property** is a subtype of **Condition_property** and **Property**. Thus it inherits all relationships from the attribute classes of its supertypes, i.e. identifying attributes, semantic attributes, administrative attributes, value attributes, and translation data.

Figure 10 shows the detailed model of the information object "**Dependent_condition_property**" using UML as presentation technique.

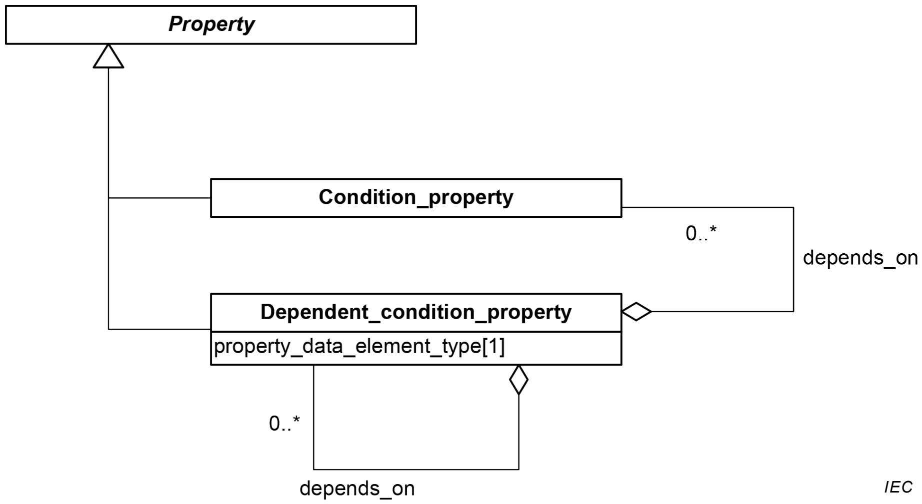

Figure 10 – Dependent_condition_property (UML class diagram)

12.2 Depends on

Attribute name:	**depends_on**
Attribute definition:	set of one or more identifier(s) of **Condition_property** information objects acting as conditions on which the characteristic depends that is expressed by the **Dependent_condition_property** information object.
Description:	Conditions are specified in detail in Clauses 11 and 12.
Obligation:	mandatory
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

12.3 Property_data_element_type

Attribute name:	**property_data_element_type**	
Attribute definition:	role of the **Property**	
Description:	One of the following roles shall be assigned:	
	DEPENDENT_C_DET	**Property** acting as dependent condition
Obligation:	mandatory	
Character type of values:	See Clause 26.	

13 Dependent_property

13.1 Specification of information object

Name:	**Dependent_property**
Definition:	property information object that is subject to restrictions expressed by one or more condition properties or dependent condition properties
Description:	any instance of **Dependent_property** shall contain the value "DEPENDENT_P_DET" in the (inherited) attribute **product_data_element_type**.

NOTE **Dependent_property** is a subtype of **Property**. Thus it inherits all relationships from the attribute classes of its supertypes, i.e. identifying attributes, semantic attributes, administrative attributes, value attributes, and translation data.

Figure 11 shows the detailed model of the information object "**Dependent_property**" using UML as presentation technique.

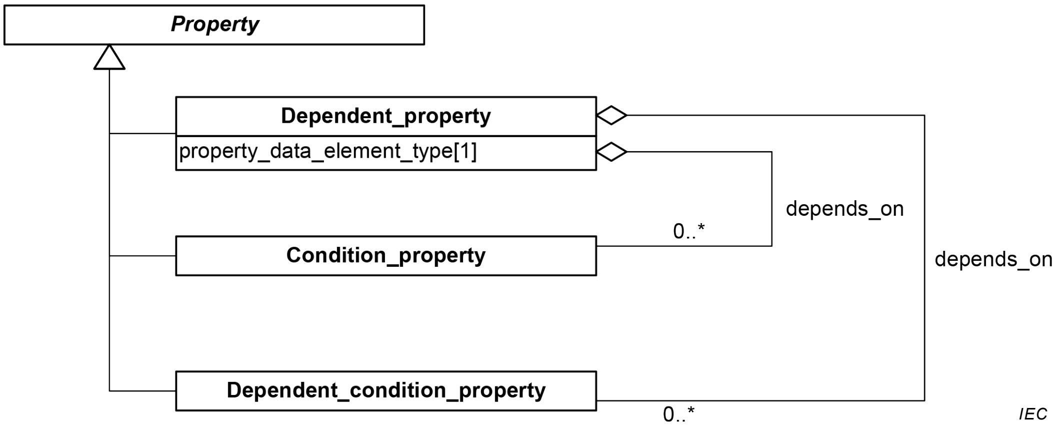

Figure 11 – Dependent_property (UML class diagram)

13.2 Depends on

Attribute name:	**depends_on**
Attribute definition:	set of one or more identifiers of conditions on which the characteristics of a dependent property or dependent condition depends
Description:	Conditions are specified in detail in Clauses 11 and 12.
Obligation:	mandatory
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

13.3 Property_data_element_type

Attribute name:	**property_data_element_type**
Attribute definition:	role of the **Property**
Description:	One of the following roles shall be assigned:
	DEPENDENT_P_DET — **Property** acting as dependent property
Obligation:	mandatory
Character type of values:	See Clause 26.

14 Non_dependent_property

14.1 Specification of information object

Name:	**Non_dependent_property**
Definition:	property information object that is not subject to any restrictions within its defined scope
Description:	any instance of **Non_dependent_property** shall contain the value "NON_DEPENDENT_P_DET" in the (inherited) attribute **product_data_element_type**.

NOTE **Non_dependent_property** is a subtype of **Property**. Thus it inherits all relationships from the attribute classes of the supertype, i.e. identifying attributes, semantic attributes, administrative attributes, value attributes, and translation data.

Figure 12 shows the detailed model of the information object "**Non_dependent_property**" using UML as presentation technique.

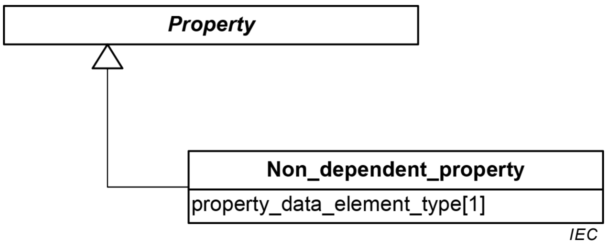

Figure 12 – Non_dependent_property (UML class diagram)

14.2 Property_data_element_type

Attribute name:	**property_data_element_type**
Attribute definition:	role of the **Property**
Description:	One of the following roles shall be assigned:

NON_DEPENDENT_P_DET	**Property** acting as non-dependent property

Obligation: mandatory

Character type of values: See Clause 26.

15 Translation_information

15.1 Specification of information object

Name: **Translation_information**

Definition: collection of attributes to manage localized content of information objects such as **Property** and **Item_class**

Description:

Figure 13 shows the detailed model of the information object "**Translation_information**" using UML as presentation technique.

Translation_information
language_code[1] translation_revision[1] date_of_current_translation_revision[0..1] responsible_translator_coded[0..1] responsible_translator[0..1]

IEC

Figure 13 – Translation_information (UML class diagram)

15.2 Date_of_current_translation_revision

Attribute name: **date_of_current_translation_revision**

Attribute definition: date of the latest revision of the translation

EXAMPLE "2005-06-28"

Obligation: optional

Character type of values: date format according to YYYY-MM-DD as specified by ISO 8601

15.3 Language_code

Attribute name: **language_code**

Attribute definition: code of the language in which the translated text is written

EXAMPLE "en GB", "en US", "de DE", "de AT"

Description: The value of **language_code** shall be taken from the two-letter language code according to ISO 639-1 followed by an intermediate blank and extended with the alpha-2 country code according to ISO 3166-1 or the country subdivision code according to ISO 3166-2 to enable the possibility to distinguish between particular variants within one language.

Obligation: mandatory

Character type of values: See Clause 26.

15.4 Responsible_translator

Attribute name: **responsible_translator**

Attribute definition: the name of the organization responsible for the translation identified by its name and its two-letter country code.

EXAMPLE "DKE DE"

Description:

Obligation: optional

Character type of values: String constant as specified

15.5 Responsible_translator_coded

Attribute name: **responsible_translator_coded**

Attribute definition: coded identification of the organization responsible for the translation

EXAMPLE "0112/2"

Description: The identifier is specified in ISO 13584-26.

Obligation: optional

Character type of values: String constant as specified

15.6 Translation_revision

Attribute name: **translation_revision**

Attribute definition: revision number of the corresponding translation to administer changes

EXAMPLE "01"

Obligation: mandatory

Character type of values: Digits "0" through "9".

16 Value_list

16.1 Specification of information object

Name:	**Value_list**
Definition:	set of representations of permissible values of a **Property**
Obligation:	conditional
Condition:	**Value_list** shall be specified for classifying property information objects. For non-classifying property information objects a **Value_list** is optional.
Description:	An enumeration data type always references a **Value_list** for specifying the set of possible values (see 10.4.3). Value lists that form a polymorphism (see 24.7) are considered to be restrictive, because these are exhaustive and no additional or modified value is possible. All other value lists may be specified as being extendable by the user. For explicitly indicating the fact of having an extended value list, the option “others” shall be added to the value list in question to indicate an extended value list. The options specifying the meaning of the "others" shall be added as a condition of the property referencing the value list. This condition shall contain the additional options as an enumeration data type. In cases when the additional options are not yet specified, this enumeration remains empty. NOTE 1 In this case the (original) enumeration type property is specified as DEPENDENT_P_DET. Such entries can be incorporated into the value list for a future release of the dictionary. Property specialization as introduced in IEC 62656-1:2014, 5.4.5 is another technique for specifying properties with value lists differing from those contained in the ontology supplied by the original provider of the dictionary. This technique avoids any interference with the content delivered by the original provider. NOTE 2 In cases where value lists have to be narrowed, **Relation** information objects can be used (see 24.8.2). NOTE 3 In cases where a value list should be permanently extended, the value range can be extended by revising the enumeration information object in question.

NOTE **Value_list** can be assigned to a **Property** via the attribute **data_type**. In case of an assigned value list this attribute contains one of the enumeration data types which include the reference to the **Value_list** (see 10.4.3).

Figure 14 shows the detailed model of the information object "**Value_list**" using UML as presentation technique.

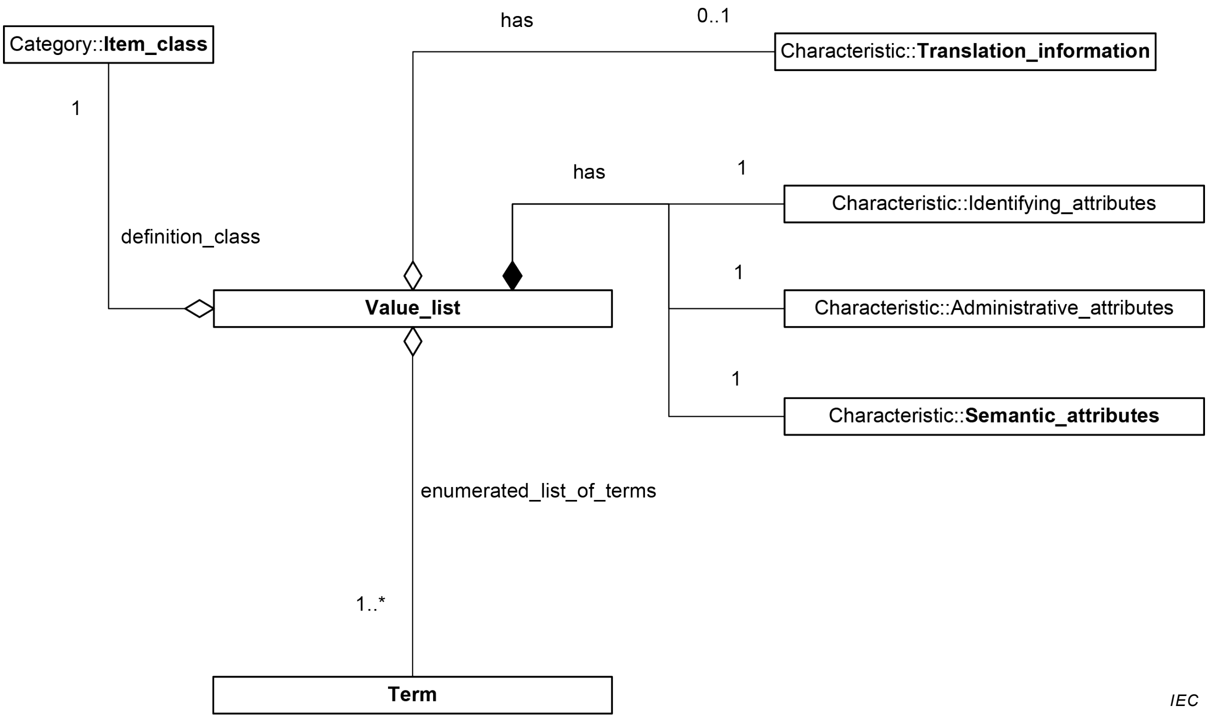

Figure 14 – Value_list (UML class diagram)

16.2 Attributes internal to Value_list

Value_list has attributes that are grouped for better readability in the information objects:

– **Identifying_attributes**;
– **Semantic_attributes**;
– **Administrative_attributes**;
– **Translation_information**.

These attribute containers are related to **Value_list** by the relationships "**has**". The information objects shall be kept together with the referencing class, i.e. they are internal to the instances of **Value_list**.

16.3 Definition_class

Attribute name:	**definition_class**
Attribute definition:	class in which the information object is defined
Description:	The class referenced by **definition_class** specifies the scope for the **Value_list** information objects. Outside of this scope the instances are invalid.
Obligation:	mandatory
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

16.4 Enumerated_list_of_terms

Attribute name: **enumerated_list_of_terms**

Attribute definition: set of references to information objects that make up in their totality the content of the value list

Description:

Obligation: mandatory

Character type of values: Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

17 Term

17.1 Specification of information object

Name: **Term**

Definition: representation of a permissible value as an element of a **value_list**

Description:

Figure 15 shows the detailed model of the information object "**Term**" using UML as presentation technique.

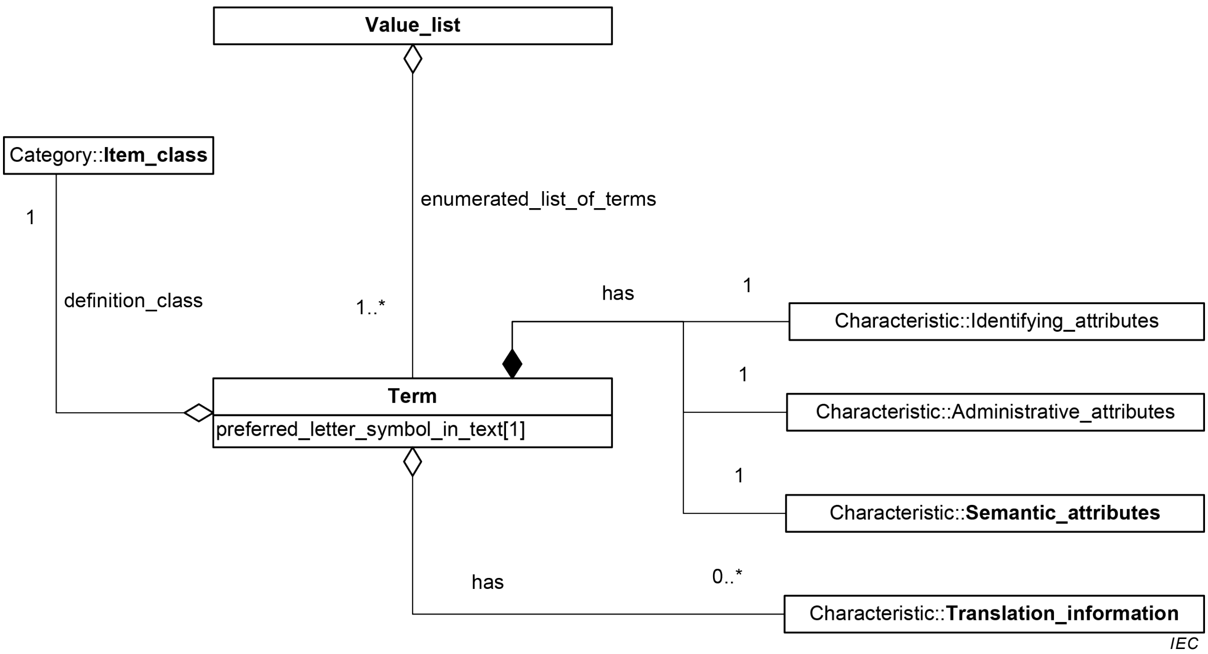

Figure 15 – Term (UML class diagram)

17.2 Preferred_letter_symbol_in_text

Attribute name:	**preferred_letter_symbol_in_text**
Attribute definition:	representation of a permissible value within a **Value_list**
Description:	If used as value code of a classifying property this value code shall be the same as the coded name of the **Item_class** which is created by that classifying property. The value code of a **Classifying_property** shall have a maximum length of 18 alphanumeric characters.
Obligation:	mandatory
Character type of values:	See Clause 26.

17.3 Attributes internal to Term

Term has attributes that are grouped for better readability in the information objects:

– **Identifying_attributes**;
– **Semantic_attributes**;
– **Administrative_attributes**;
– **Translation_information**.

These attribute containers are related to **Term** by the relationship "**has**". The information objects shall be kept together with the referencing class, i.e. they are internal to the instances of **Term**.

17.4 Definition_class

Attribute name:	**definition_class**
Attribute definition:	class in which the item is defined
Description:	The class referenced by **definition_class** specifies the scope for the **Term** information object. Outside of this scope the instances are invalid.
Obligation:	mandatory
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

18 Drawing

18.1 Information model

In Clause 18, the various attributes of a **Drawing** are explained. For a list of these attributes, see Table 5. These attributes are related to the identification of drawings.

Table 5 – List of attributes of Drawing

Attribute names in IEC 61360-1	Attribute names in IEC 61360-2	Subclause number
code	Code	18.3
date_of_current_translation_revision	[a]	15.2
descriptive_designator	[a]	18.4
drawing_title	[a]	18.5
file_format	[a]	18.6
file_name	[a]	18.7
language_code	[a]	15.3
obsolete_date	[a]	9.3
proposal_date	date_of_original_definition	9.4
published_by	[a]	9.6
published_in	[a]	9.5
responsible_translator	[a]	15.4
responsible_translator_coded	[a]	15.5
revision_number	Revision	18.8
status_level	[a]	9.9
translation_revision	[a]	15.6
version_initiated_on	[a]	9.10
version_number	Version	18.9
version_released_on	date_of_current_version	9.11
[a] Not defined in IEC 61360-2.		

Figure 16 shows the overview model of the module "Drawing concept" using UML as presentation technique.

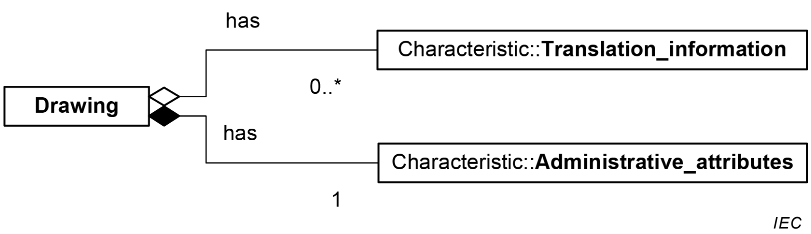

Figure 16 – Overview model for Drawing concept (UML class diagram)

18.2 Specification of information object

Name: **Drawing**

Definition: collection of identifying and administrative attributes to allow management and use of data containing pictorial information

Description:

Figure 17 shows the detailed model of the information object "**Drawing**" using UML as presentation technique.

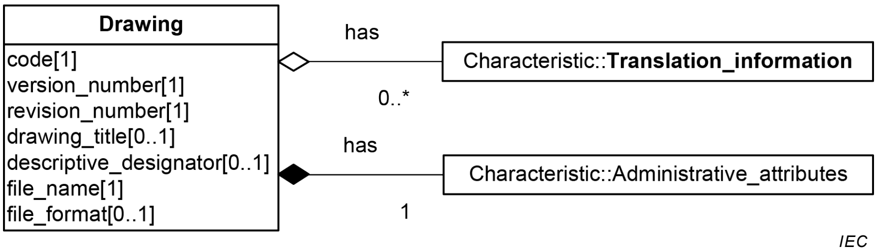

Figure 17 – Drawing (UML class diagram)

18.3 Code

Attribute name: **code**

Attribute definition: unique six-character code for a **Drawing**

NOTE IEC 61360-2:2012 specifies 14 characters as the maximum length of the attribute **code**. The difference is caused by information appended to the code such as the International Registration Data Identifier (see ISO/IEC 11179-6:2005).

Description: The first three characters shall be alphabetic, the last three numeric (format AAANNN). The character "X"[4] shall not be used as the first character. The codes are issued sequentially and should not have any relationship with the meaning of the drawing.

Obligation: mandatory

Character type of values: Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

18.4 Descriptive_designator

Attribute name: **descriptive_designator**

Attribute definition: coded representation of the **drawing_title**

4 For local use, all codes starting with XAA up to and including codes starting with XZZ may be used and are therefore excluded from the set of standard drawings.

Description: The descriptive designator is used to identify the content of a particular **Drawing**.

To ensure consistency and comprehensibility of **descriptive_designator**, coding systems may be used. The example demonstrates the use of a coding system to specify the content of drawings for semiconductor device packages.

EXAMPLE The descriptive designator has the form XYY-Z-Tnnn-Bmmm, where the characters have the following meanings:

– X indicates the disposition of the terminals, defined by the relevant classifying property;

– YY indicates the semiconductor device package style of the component, defined by the relevant classifying property;

– Z indicates the shape of the terminals, defined by the relevant classifying property;

– Tnnn is the terminal variant code, defined by the relevant classifying property;

– Bmmm is the optional body variant code, defined by the relevant classifying property.

The **descriptive_designator** of a **Drawing** shall have a maximum length of 17 alphanumeric characters.

Obligation: optional

Character type of values: See Clause 26.

18.5 Drawing_title

Attribute name: **drawing_title**

Attribute definition: single-word or multi-word descriptive caption applied to a drawing

Description: The **drawing_title** shall have a maximum of 70 characters

Obligation: mandatory

Character type of values: See Clause 26.

18.6 File_format

Attribute name: **file_format**

Attribute definition: the data format in which the drawing is represented

Description: More than one data format for a drawing is allowed [5]

If the file format specified by the attribute **file_format** contradicts the file format information derived from attribute **file_name**, the format given in the attribute **file_format** shall have priority.

Obligation: mandatory

18.7 File_name

Attribute name: **file_name**

Attribute definition: the name of the file on a carrier which contains the **Drawing** data

Description: The **file_name** shall be complete, including the **file_name** extension.

If the file is stored on a remote server, the complete path of the URL (including the **file_name**) shall be provided.

Obligation: mandatory

18.8 Revision_number

Attribute name: **revision_number**

Attribute definition: number used for administrative control of a **Drawing**

Description: The **revision_number** of a **Drawing** should consist of 3 digits. Although the common ISO/IEC EXPRESS information model allows more digits, for the dictionary to be registered into IEC CDD or to be compatible with IEC 61360 series only 3 digits should be used. For specific purposes, revision numbers with more digits may be used. Consecutive revision numbers starting from "000" shall be issued in ascending order. Per **Drawing**, unique by its identifier, only one revision number is current at any moment.

A new revision of a **Drawing** shall be generated if the drawing is changed without affecting either its use or its meaning, or when corrections of typing or spelling errors have been implemented.

The **revision_number** shall be reset to its starting value "000" when the **version_number** changes.

Obligation: mandatory

---

5 It is the responsibility of the sender of a drawing to clarify the data format used. Possible formats include bit-map formats such as BMP, TIFF, GIF and JPEG, vector formats such as CGM, DXF and HP-GL and document formats such as PDF.

Character type of values:	Digits "0" through "9".

18.9 Version_number

Attribute name:	**version_number**
Attribute definition:	number used to indicate the versions of a **Drawing** during its life cycle
Description:	The **version_number** of a drawing should consist of 3 digits. Although the common ISO/IEC EXPRESS information model allows more digits, for the drawing to be registered into IEC CDD or to be compatible with IEC 61360 series only 3 digits should be used. For specific purposes, version numbers with more digits may be used. Consecutive version numbers starting from "000" shall be issued in ascending order. Per **Drawing**, unique by its identifier, only one **version_number** is current at any time. The **version_number** of a **Drawing** shall be changed when there is a major change such as the addition of a new dimension parameter. A change to a **Drawing** version should result in a change to the version of the information object that references the **Drawing**.
Obligation:	mandatory
Character type of values:	Digits "0" through "9".

18.10 Attributes internal to Drawing

Drawing has attributes that are grouped for better readability in the information objects:

– **Administrative_attributes**; and

– **Translation_information**.

These attribute containers are related to **Drawing** by the relationship "**has**". The information objects shall be kept together with the referencing class, i.e. they are internal to the instances of **Drawing**.

19 Unit of measure

19.1 Overview

Quantitative **Property** information objects require information about their physical quantity and the applicable unit(s). Without the unit of measure, any assigned values are meaningless and serious errors can occur.

Attribute **data_element_type_class** (see 6.3) specifies the physical quantity of the **Property** information object. Thus preselecting suitable units is supported and the identification and assignment of the unit of measure is much simplified.

NOTE 1 Annex B lists use and available type classification codes of properties.

The unit of measure may be specified by using the codes specified in IEC TS 62720 or compliant unit dictionaries.

NOTE 2 IEC maintains in IEC CDD an IEC TS 62720 compliant collection of some 1 500 units grouped as physical quantities readily available for use with **Property** information objects. The **Property** attributes **code_of_unit** and **code_of_alternative_unit** allow referencing these information objects.

Additionally, system of quantities is available through **Relation** and its attribute **role_of_the_relation** = QUANTITY (see 22.7). Thus, the existing systems of units may be implemented.

NOTE 3 Currently the system(s) of units are implemented via the IEC CDD information object "List of units" which is a predecessor of the above referenced **Relation** information object.

NOTE 4 Even though being designed as integral part of IEC 61360 series compliant environments, instantiated **Unit_of_measure** information objects can also be referenced via ICID from environments other than IEC 61360 series for providing unambiguous information about unit of measure.Figure 18 shows the overview model of the module "Measure concept" using UML as presentation technique.

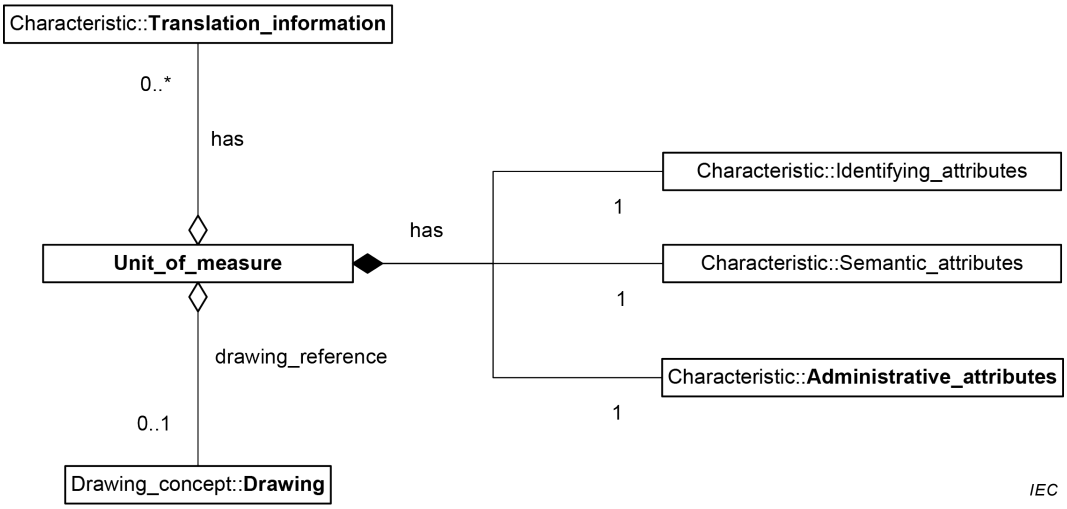

Figure 18 – Overview model for Measure concept (UML class diagram)

19.2 Specification of information object

Name: **Unit_of_measure**

Definition: definite magnitude of a physical quantity, defined and adopted by convention or by law, that is used as a standard for measurement of the same physical quantity

EXAMPLE metre, second, degree Celsius, etc.

Description:

Figure 19 shows the detailed model of the information object "**Unit_of_measure**" using UML as presentation technique.

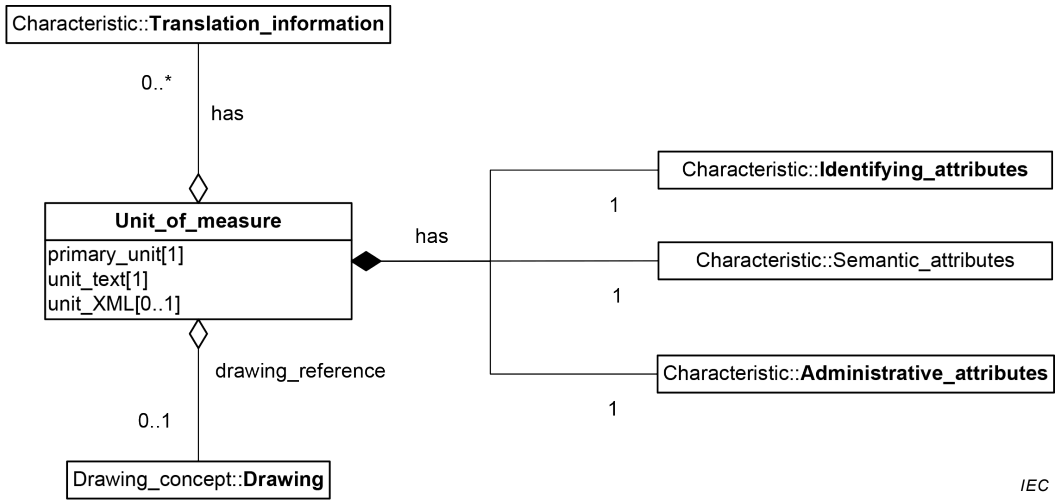

Figure 19 – Unit_of_measure (UML class diagram)

19.3 Primary_unit

Attribute name:	**primary_unit**
Attribute definition:	unit expressed with SI base units
Description:	
Obligation:	mandatory
Character type of values:	See Clause 26.

19.4 Unit_in_text

Attribute name:	**unit_in_text**
Attribute definition:	textual presentation of the unit
Description:	Units may comprise sophisticated mathematical formulae, thus the expression presented here may deviate from the mathematical correct presentation of the unit
Obligation:	mandatory
Character type of values:	See Clause 26.

19.5 Unit_XML

Attribute name:	**unit_XML**
Attribute definition:	presentation of the unit expressed with xml code
Description:	

Obligation:	optional
Character type of values:	characters in accordance with the xml specification

19.6 Attributes internal to Unit_of_measure

Unit_of_measure has attributes that are grouped for better readability in the information objects:

– **Administrative_attributes**;
– **Identifying_attributes**;
– **Semantic_attributes**;
– **Translation_information**.

These attribute containers are related to **Unit_of_measure** by the relationships with the name "**has**". The information objects shall be kept together with the referencing class, i.e. they are internal to the instances of **Unit_of_measure**.

19.7 Drawing_reference

Attribute name:	**drawing_reference**
Attribute definition:	reference to the identifier of a drawing which contains a pictorial representation of the concept
Comments	
Obligation:	optional
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

20 Creation of language variants

20.1 Overview

Figure 13 shows the attributes that contain additional administrative information about localized properties or classes. These attributes shall be maintained for each localized information object.

An information object in a language other than English shall only be prepared if previously the released English reference version is available.

NOTE Due to delays in the preparation of national language versions, users can face inconsistencies between the English reference version and additional language variants.

20.2 Language-dependent attributes of a Property

The following attributes of a **Property** are language-dependent and may be translated:

– **definition**;
– **note**;
– **preferred_name**;
– **remark**;

– **short_name**;
– **synonymous_name**.

20.3 Language-dependent attributes of an Item_class

The following attributes of an **Item_class** are language-dependent and may be translated:

– **definition**;
– **note**;
– **preferred_name**;
– **remark**;
– **short_name**;
– **synonymous_name**.

20.4 Language-dependent attributes of a Drawing

The following attributes of a **Drawing** are language-dependent and may be translated:

– **drawing_title**.

20.5 Language-dependent attributes of a Unit_of_measure

The following attributes of a **Unit_of_measure** are language-dependent and may be translated:

– **definition**;
– **note**;
– **preferred_name**;
– **remark**;
– **short_name**;
– **synonymous_name**.

20.6 Language-dependent attributes of a Relation

The following attributes of a **Relation** are language-dependent and may be translated:

– **definition**;
– **note**;
– **preferred_name**;
– **remark**;
– **short_name**;
– **synonymous_name**.

21 Item_class

21.1 Classification tree

For the classification of items, the principle of dividing a set of items into subsets may be applied repeatedly, thereby creating a hierarchical tree of classes using so-called "is-a" structures. The tree shall start with the root class and consists further of superclasses and subclasses.

Classification principles are:

– a superclass should have two or more subclasses;

– a superclass should have one classifying property defining these subclasses;
– a subclass shall have exactly one superclass;
– a subclass becomes a superclass when it gets subclasses.

The topmost class (root class) of a classification tree has no superclass. This class has only subclass information objects. This topmost class shall carry the value "UNIVERSE" as **code** and shall neither be involved in any relationship nor reference any other information object.

NOTE 1 The union of two universe nodes is still a universe.

NOTE 2 Universe is a virtual class signifying the Universal Class in mathematical logic: the class of all classes.

The goal of a classification is to split the confusing set of items in the examined domain into smaller, more manageable sets that may be considered separately.

EXAMPLE One of the most successful classifications, still in use, is the classification of the kingdom of animals set up by Carl Linnaeus in the year 1735.

A property which is applicable to a class shall also be applicable to all subclasses of this class. Properties which are only applicable for a limited number of subclasses have to be specified in the relevant subclass(es).

In this way, a tree structure is created with superclasses and subclasses as shown in Figure 20. In this figure, each class shows, as dots, the collection of properties which are applicable for this class. However, the set of applicable properties in a class includes also the properties inherited from higher level classes (superclasses). So where a class is shown as empty, it nevertheless includes all inherited property information objects.

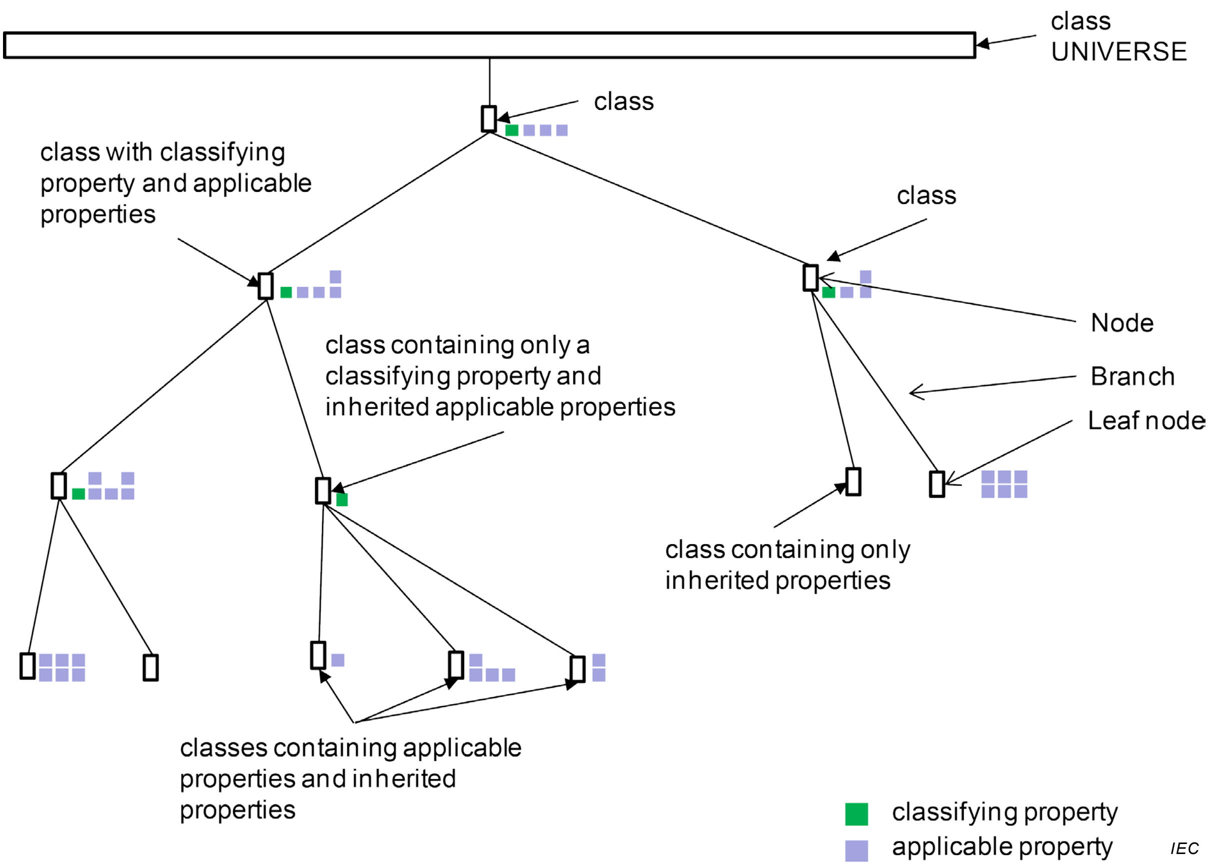

Figure 20 – Classification tree

The classification of the items described here is set up along the following lines.

a) The hierarchical order of the classes is determined by specialization, meaning that a class always comprises its next lower level subclass(es). The classification ends at the level where no further division of items shall be made.

b) From the collection of properties applicable at a particular node, one property is the classifying one.

c) The classifying properties, applicable to all classes with subclasses in the tree, shall have the following characteristics:

   – each one comprises a single elementary attribute and not a combination of different attributes, i.e. they have homogeneous value lists;

   – the subclasses designated by the values of a classifying property are complementary; they fully cover in their totality the whole field specified by their superclass without overlap. So-called multiple value fields, where two or more values apply at the same time, are not allowed;

   – the value codes of classifying properties shall be unique within the whole classification scheme;

   – no garbage classes are allowed, such as "miscellaneous", "remaining", "various";

   – any relevant attributes may be used for classification; their choice and order are determined by whether or not they lead to specific properties which are relevant to the users;

   – the classification criteria used are mostly related to functionality, but other criteria may occur such as technology, application, material, or geometry; the choice may vary within a classification tree between items of different type;

   – classifying properties have always the data type value: "ENUM_CODE_TYPE".

d) For all classifying properties, visibility and applicability is ensured:

   – a property is specified as applicable for a class if it is explicitly associated to that class. It shall satisfy an essential characteristic of the items represented by that class;

   – a condition becomes applicable to a class when it is referenced by one of the properties relating to the class.

e) The terms which relate to the branches of the tree, being a value of the appropriate classifying property, shall have the following characteristics:

   – they are precise – no vague or ambiguous terms such as "general purpose", "economical", "high speed", etc. are allowed;

   – they have a clearly defined meaning;

   – they conform to an international standard where available;

   – they are free from synonymy – where synonyms do occur, one term shall be selected as the preferred name and the others refer to it;

   – the specific meaning of homonyms shall be explained by context indications.

When the content of IEC CDD is extended, new sets of classes and properties have to be added to IEC CDD. Four approaches may be applied.

1) Adding individual classes and properties at various points within the existing class tree of IEC CDD.

2) Adding complete branches to the IEC CDD class tree(s). Within the branches the new classes shall have domain specific structures suitable for the concepts characterized.

3) Adding complete tree(s) to the IEC CDD data structure that completely cover the domain in question. In many cases domains (re-)use concepts from neighbouring domains; for example, the domain of process automation can embrace capacitors, which are contained in the domain of electronic components. To avoid duplicated classes or properties, the class "capacitors" should be imported using the case-of relationship.

4) Adding complete tree(s) without any specific relation to already existing classes. This case is not desirable because duplicates of classes or properties in the IEC CDD data can

occur. Such trees should be subjected to revision and migrated to approach 3) in a mid-term timeframe.

When extending the dictionary with content of a new domain the UNIVERSE class plays an important role. The UNIVERSE class has no restrictions by definition. Thus, the UNIVERSE class embraces all other classes by definition and any new trees shall have this class as topmost class.

Table 6 – List of attributes of Item_class as defined in IEC 61360-1 and their equivalent in IEC 61360-2

Attribute names in IEC 61360-1	Attribute names in IEC 61360-2	Subclause number
applicable_data_element_type	described_by	21.9
applicable_relation	[a]	21.10
classifying_data_element_type	[a]	21.11
class_type	[a]	21.6
code	code	7.3
coded_name	short_name	21.7
definition	definition	8.3
drawing_reference	simplified_drawing	21.12
note	note	8.4
obsolete_date	is_deprecated/ date_of_current_version	9.3
preferred_name	preferred_name	7.4
proposal_date	date_of_original_definition	9.4
published_in	[a]	9.5
published_by	[a]	9.6
remark	remark	8.5
responsible_committee	[a]	9.7
revision_number	revision	7.5
revision_released_on		9.8
source_document_of_definition	source_doc_of_definition	8.6
status_level	[a]	9.9
superclass	its_superclass	21.14
synonymous_name	synonymous_name	7.7
version_initiated_on	[a]	9.10
version_number	version	7.8
version_released_on	date_of_current_version	9.11
[a] Not defined in IEC 61360-2:2012.		

21.2 Composition tree

A composition tree expresses the part-whole structure between an item and its members such as between a product and its parts or between an information collection and its constituents.

EXAMPLE 1 The bill of material of a part represents a composition tree.

These relationships between an item and its members are called "has-a" relationships. The "has-a" relationship is implemented by a **Property** information object of type "CLASS_INSTANCE_TYPE" or "CLASS_REFERENCE_TYPE" (see 10.4.4 and 10.4.5).

In Figure 21 A100 is made up of two constituents B110 and B120, whereas B120 is again made from C111 and C112. There is no inheritance within composition trees.

EXAMPLE 2 In Figure 21 A100 can be a flow meter consisting of two modules, an input interface and an output interface, the output interface offering two configurations: 20 mA interface (C111) and 0-24 V interface (C112).

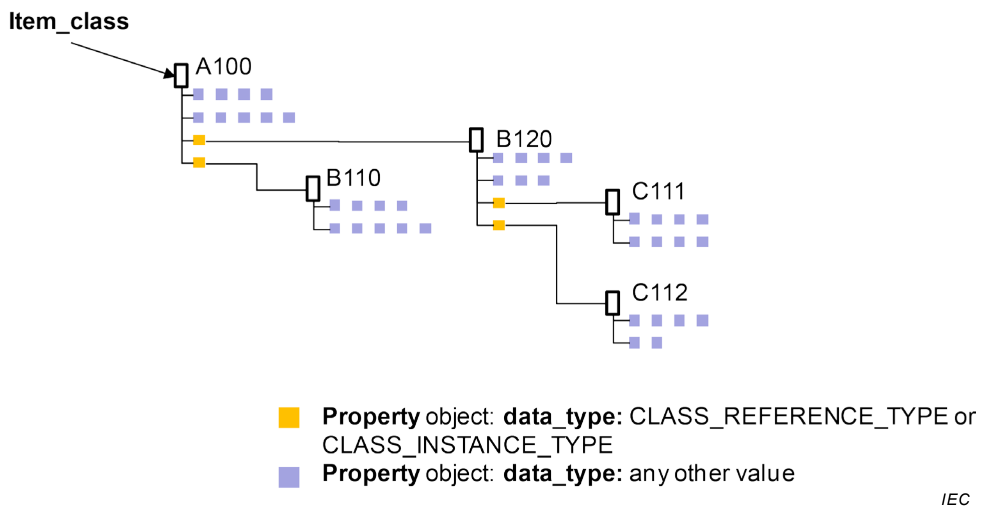

Figure 21 – Composition tree

Table 6 lists the various attributes of classes as encountered in the specifications. These attributes are related to identification, description and to relationships.

21.3 Use of auxiliary schemes for classification and coding of values

To ensure consistency and comprehensibility of properties, the use of coding systems and classification schemes for definitional parts and associated values is recommended. The example demonstrates the use of a classification scheme to specify the shapes of semiconductor device package outlines.

NOTE When used, auxiliary schemes are referenced in the "**note**" attribute.

EXAMPLE The classification scheme for the semiconductor device package outlines is based on the following three characteristics of the semiconductor device package according to the codes defined in IEC 60191-4:

– outline style;

– terminal position;

– terminal form.

Based on these characteristics, five levels of classification are defined as follows:

– level 1: outline style;

– level 2: terminal position;

– level 3: terminal form;

– level 4: terminal variant;

– level 5: body variant (optional).

The specification of each class at the lowest level of the classification tree of the semiconductor device package contains an attribute which makes a reference to a unique drawing graphically specifying the dimension parameters. See IEC 60191-4:2013, Clause 4 for the description of the parameters.

21.4 Information model

Figure 22 shows the model of the information object "**Concept_data**".

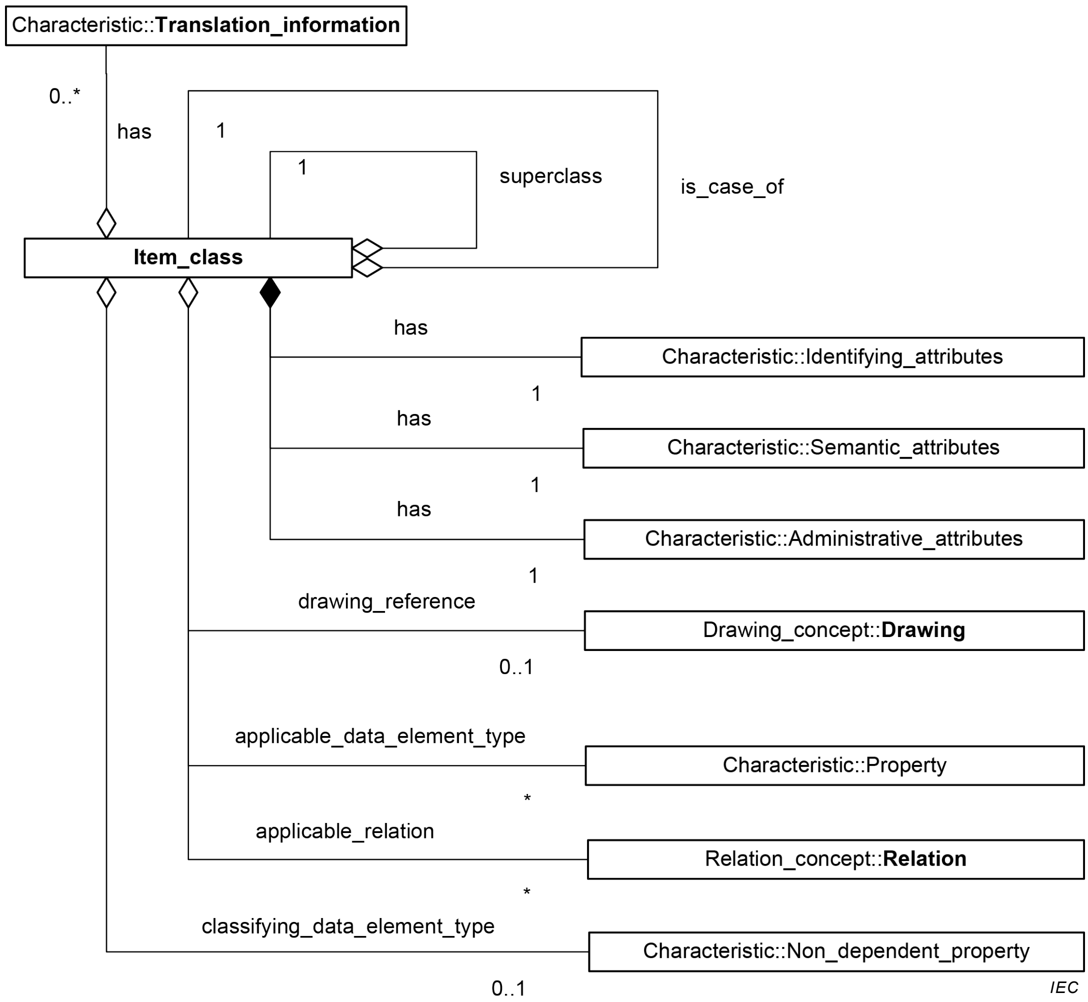

Figure 22 – Overview model for Concept_data (UML class diagram)

21.5 Specification of information object

Name: **Item_class**

Definition: description of a set of information objects that share the same characteristics

Description:

Figure 23 shows the detailed model of the information object "**Item_class**" using UML as presentation technique.

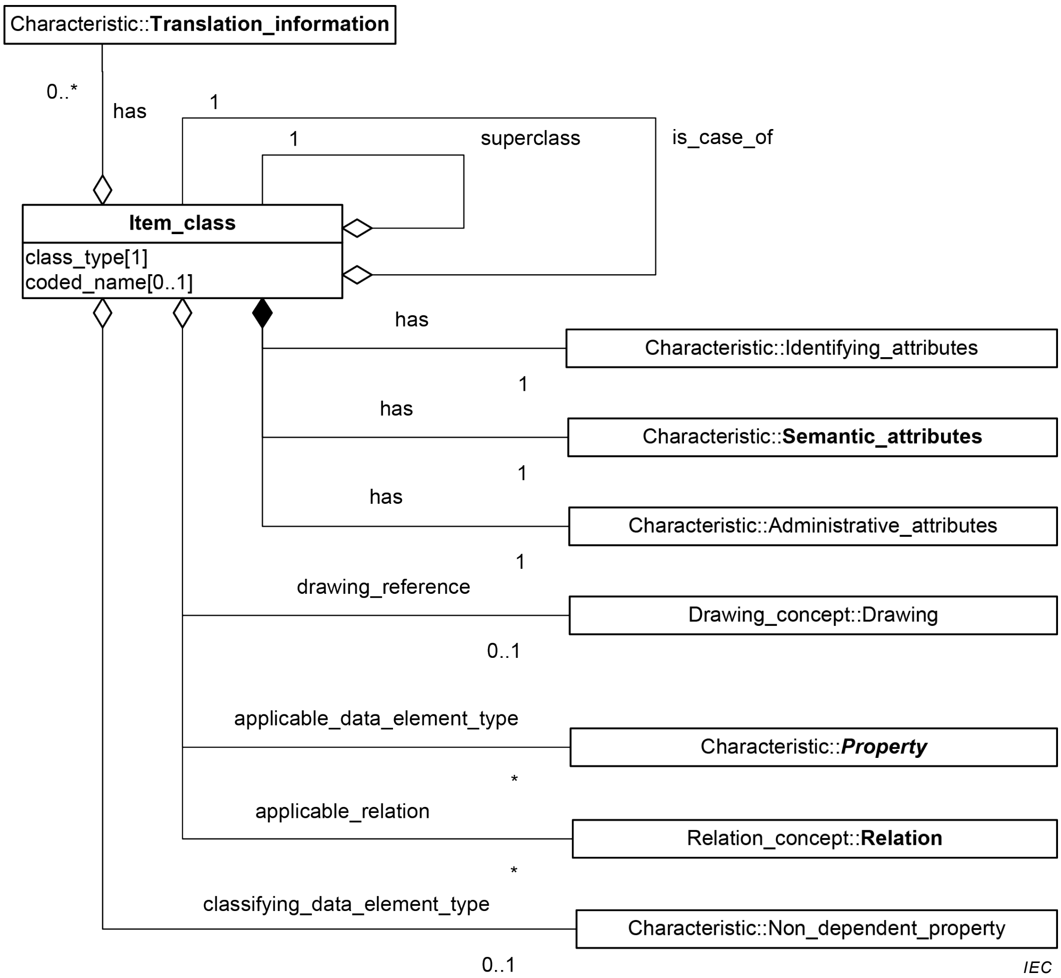

Figure 23 – Item_class (UML class diagram)

21.6 Class_type

Attribute name:	**class_type**	
Attribute definition:	additional information about the type of hierarchy containing the class	
Description:	Allowed values:	
	ITEM_CLASS	class is part of an is-a hierarchy and related to its superclass by inheritance (see IEC 61360-2:2012, 5.8)
	CATEGORICAL_ CLASS	class is part of a categorization hierarchy and related to its superclass by **is_case_of** (see IEC 61360-2:2012, 5.8)

	FEATURE_CLASS	All classes below class "AAA233 – Features" represent commonly used concepts that may be used independently or be referenced from other classes. CLASS_INSTANCE_REFERENCE should be used if referencing is applied.
Obligation:	mandatory	
Character type of values:	See Clause 26.	

NOTE COMPONENT_CLASS and MATERIAL_CLASS were defined within the dictionary model of IEC 61360-1:1995. These types are deprecated, and they are removed from this document. (See also IEC 61360-2:2012, 5.8.1.)

21.7 Coded_name

Attribute name:	**coded_name**
Attribute definition:	coded representation of the **preferred_name** which shall be equal to the corresponding value code (see 21.7) of the classifying property
Description:	The first character of the **coded_name** shall be a letter. The **coded_name** of an **Item_class** shall have a maximum length of 18 alphanumeric characters
Obligation:	mandatory
Character type of values:	See Clause 26.

21.8 Attributes internal to Item_class

Item_class has attributes that are grouped for better readability in the information objects:

– **Administrative_attributes**;
– **Identifying_attributes**;
– **Semantic_attributes**;
– **Translation_information**.

These attribute containers are related to **Item_class** by the relationship "**has**". The information objects shall be kept together with the referencing class, i.e. they are internal to the instances of **Item_class**.

21.9 Applicable_data_element_type

Attribute name:	**applicable_data_element_type**

Attribute definition:	applicable property necessarily possessed by each part that is member of a characterization class NOTE 1 Each part that is member of a characterization class possesses an aspect corresponding to each applicable property of this characterization class. NOTE 2 The above definition is conceptual; there is no requirement for using all applicable properties of a class for describing each part of this class at the data model level. NOTE 3 All the applicable properties of a superclass are also applicable properties for the subclasses of this superclass. NOTE 4 Only properties defined or inherited as visible and imported properties of a class can be applicable properties. [The definition and Notes are taken from IEC 61360-2:2012, 3.2]
Obligation:	conditional
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

Being listed as **applicable_data_element_type** or being inherited from a supertype does not imply that a specific **Property** is optional or mandatory. There is no obligation for the user to provide values for any **Property** information object.

NOTE 5 Contracts between the owner of a database and the user(s) can stipulate the existence of mandatory content within the database in question.

21.10 Applicable_relation

Attribute name:	**applicable_relation**
Attribute definition:	**Relation** defined for an **Item_class**
Obligation:	optional
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

Being listed as **applicable_relation** does not imply that a specific **Relation** is optional or mandatory.

NOTE Contracts between the owner of a database and the user(s) can stipulate the existence of mandatory content within the database in question.

This attribute is not subject to inheritance.

21.11 Classifying_data_element_type

Attribute name:	**classifying_data_element_type**
Attribute definition:	**Value_list** applicable for a particular **Item_class**, addressing a single elementary property having a value list, whose values define the subclasses

Description:	The value codes (i.e. **preferred_letter_symbol_in_text**, see 17.2) of the values shall be the same as the **coded_names** (see 21.7) of the subclasses. It is also desirable that the **preferred_name** of a subclass is the same as the **preferred_name** associated with the value code. See Annex E for additional information.
Obligation:	optional
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

21.12 Drawing_reference

Attribute name:	**drawing_reference**
Attribute definition:	reference to the identifier of a **Drawing** which illustrates the meaning of an **Item_class**
Description:	The identifier of a **Drawing** shall consist of the combination of the six-character drawing code, followed by a hyphen followed by the three digit version number of the drawing references.
Obligation:	optional
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

21.13 Is_case_of

Attribute name:	**is_case_of**
Attribute definition:	(other) class that partially or fully conforms to the specification of the class
Description:	**is_case_of** allows the reuse of already defined classes.
Obligation:	optional
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

21.14 Superclass

Attribute name:	**superclass**
Attribute definition:	class that is designated as the canonical parent class of the class
Description:	If no higher level **Item_class** information object exists the term "UNIVERSE" shall be used.

Obligation:	mandatory
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

22 Relation

22.1 Overview

Relation is used to describe a named relation with a globally unique identifier. **Relation** embraces two categories; one is called PREDICATION, and the other is called FUNCTION. The difference between them is that the PREDICATION has a domain but no codomain, whilst the FUNCTION shall have domain and codomain.

NOTE When a relation R exists among $S_1, S_2, \ldots, S_n$ then R can be mathematically equated as a set R that is a subset of the Cartesian product of the n sets, i.e. $R \subseteq S_1 \times S_2 \times \ldots \times S_n$. Of course, from a mathematical point of view, a function as well as a predication is a kind of relation, and thus the basic characteristics of the relation are maintained in both the function and predication.

Different relationships among information objects may be modelled as instances of relation:

a) constraint on one or more properties, as an extension of a single property constraint;
b) grouping mechanism among information objects, in particular among heterogeneous ones;
c) mapping, correspondence, or transition among information objects.

Each role is designated by the attribute **role_of_the_relation**.

Figure 24 shows the overview model of the module "Relation_concept" using UML as presentation technique.

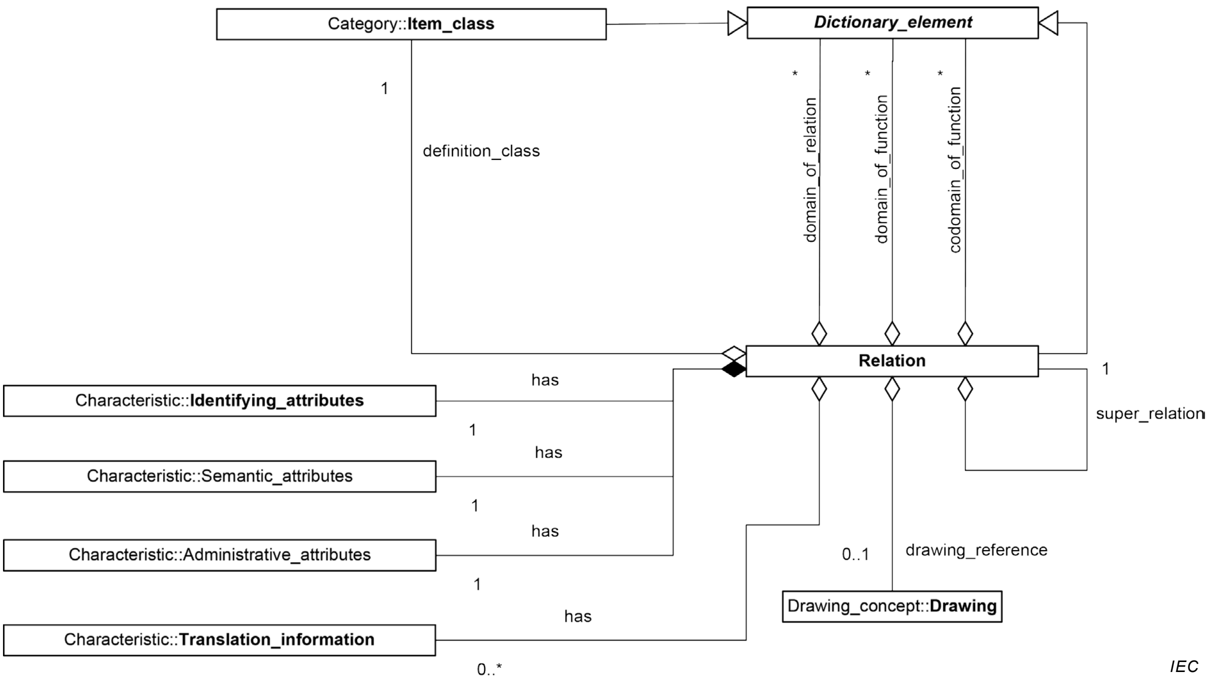

Figure 24 – Overview model for Relation concept (UML class diagram)

22.2 Specification of information object

Name: **Relation**

Definition: information object connecting two or more instances of type class or property for the purpose of applying specific side conditions

Description:

Figure 25 shows the detailed model of the information object "**Relation**" using UML as presentation technique.

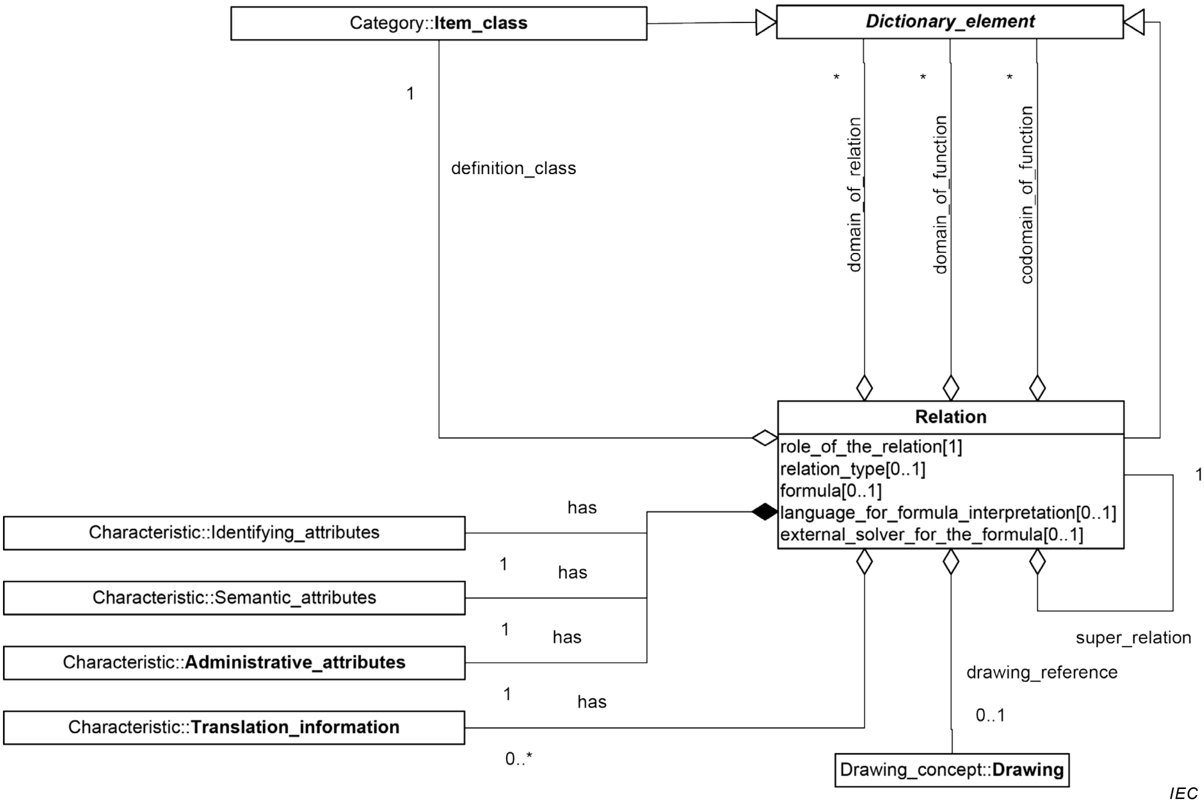

Figure 25 – Relation (UML class diagram)

22.3 External_solver_for_the_formula

Attribute name: **external_solver_for_the_formula**

Attribute definition: reference to an external solver of content of **formula** that is passed to the environment of the parcel system and from which the value of the function is returned

Description:

Obligation: optional

Character type of values: See Clause 26.

22.4 Formula

Attribute name:	**formula**
Attribute definition:	logical or mathematical formula by which the constraining effect on the domain or on the value of the codomain, in case of a functional relation, or on both may be computed

enum_constraint (*value1*, *value2*, ...) [a]	ENUMERATION
string_pattern_constraint (*pattern*) [b]	STRING_TYPE
string_range_constraint (*min*, *max*) [c]	STRING_TYPE
range_constraint (*min*, *max*, OPTIONAL *min_inclusive*, OPTIONAL *max_inclusive* [d], OPTIONAL *complement* [e])	INT, REAL, RATIONAL
cardinality_constraint (*bound1*, *bound2*) [f]	Aggregate types
subclass_constraint (*icid1*, *icid2*,...) [g]	CLASS_REFERENCE, or CLASS_INSTANCE
subtype_constraint (*type1*, *type2*,...) [h]	ENTITY INSTANCE

[a] *value1*, *value2*: one of the elements of an enumeration

[b] *pattern*: a regular expression

[c] *min*, *max*: INTEGER

[d] *min_inclusive*, *max_inclusive*: BOOLEAN, its default value is TRUE

[e] *Complement*: BOOLEAN, its default value is FALSE

[f] *bound1*, *bound2*: cardinality boundary

[g] *icid1*, *icid2*, ...: a list of ICIDs of classes which the CLASS_REFERENCE or CLASS_INSTANCE type property is allowed to reference

[h] *type1*, *type2*,... data types as defined in ISO 10303-11:2004

Description:	
Obligation:	optional
Character type of values:	See Clause 26.

22.5 Language_for_formula_interpretation

Attribute name:	**language_for_formula_interpretation**
Attribute definition:	computer language or logical description system by which the content of **formula** can be interpreted
Description:	
Obligation:	optional
Character type of values:	See Clause 26.

22.6 Relation type

Attribute name:	**relation_type**	
Attribute definition:	type of the relation	
	allowed values:	
	FUNCTION	relation *f* such that for any entity *a* there is exactly one entity *b* to which *a* is related by *f*. [SOURCE: IEC 60050-103:2009, 103-01-01, modified – Notes omitted]
	PREDICATION	Boolean-valued function P: X→ {true, false} that amounts to the characteristic function or the indicator function of the relation
Description:		
Obligation:	mandatory	
Character type of values:	See Clause 26.	

22.7 Role_of_the_relation

Attribute name:	**role_of_the_relation**
Attribute definition:	specific use of the relation
	Allowed values:

ARROW	**Relation** represents a transition from one collection of information objects to another collection of (different) information objects. The source shall be specified in **domain_of_the_function** and the destination shall be designated in the **codomain_of_the function**. **relation_type** shall be set to “FUNCTION”. NOTE 1 Attribute **codomain_of_the_function** can be empty.
PACKAGE	The information objects listed in **domain_of_the_function** are information objects that are considered to be of the same kind and belonging together. **relation_type** shall be set to “FUNCTION”. Input is **domain_of_function**, the package is always referenced in **codomain_of_function**. EXAMPLE 1 A "package" used by the object-oriented computer language “Ada”. EXAMPLE 2 “Module” in “Modula-2” and “C++”.
CONSTRAINT	IEC 61360-2/ISO 13584-42 compliant value constraint. The relation is used to model a numeric constraint on a property or on a set of properties. **relation_type** shall be set to “PREDICATION". NOTE 2 In the CONSTRAINT case, relation is assigned to the **Item_class** via the attribute **applicable_relation**.

QUANTITY

Relation is used to model the concept of a physical quantity comprising a set of units of measurement.

relation_type shall be set to "PREDICATION" or "FUNCTION".

The attribute **domain_of_the_relation** collects the IDs of units of measurement.

EXAMPLE 3 Units "millimetre", "micrometre", and "nanometre", etc., including "metre", to cover the quantity "SI length measure".

The attribute **codomain_of_the_function** contains the ID of the coherent SI unit.

NOTE 3 **codomain_of_the_function** can be empty.

EXAMPLE 4 "metre" as coherent SI unit of length measure.

The **role_of_the_relation** as a quantity may be used to model a hierarchy among quantities. The attribute **super_relation** then contains the link to the next higher level relation.

EXAMPLE 5 One **Relation** information object can comprise all length measures of SI units such as "metre", "millimetre", "micrometre", or "nanometre". The other relation information object comprises Imperial length measures with Imperial units, such as "mile", "yard", "foot", or "inch". A third relation information object which is referenced by the content of the "super relation" attributes of the "SI relation" and the "Imperial relation" represents the generic quantity "length measure" comprising SI length measure and Imperial length measure units.

NOTE 4 Future editions of IEC 61360-1 might extend this list of keywords.

NOTE 5 Other standards building upon this document might extend this list of keywords in a way which conforms to the semantic of the relation.

Description:

Obligation: mandatory

Character type of values: See Clause 26.

22.8 Attributes internal to Relation

Relation has attributes that are grouped for better readability in the information objects:

– **Administrative_attributes**;
– **Identifying_attributes**;
– **Semantic_attributes**;
– **Translation_information**.

These attribute containers are related to **Relation** by the relationship "**has**". The information objects shall be kept together with the referencing class, i.e. they are internal to the instances of **Relation**.

22.9 Definition_class

Attribute name:	**definition_class**
Attribute definition:	**Item_class** that has a scope in which the relation is meaningful
Obligation:	mandatory
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

22.10 Codomain_of_function

Attribute name:	**codomain_of_function**
Attribute definition:	ID of the property or class information object(s) that serve as codomain (value range) of a function NOTE 1 What possibly can come out of a function is called the codomain. What actually comes out of a function is called the range. NOTE 2 **codomain_of_function** can be empty.
Description:	
Obligation:	conditional
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".

22.11 Domain_of_function

Attribute name:	**domain_of_function**
Attribute definition:	list of IDs of property or class information object(s) that serve as domain for the functional relation identified by **Relation** NOTE What can go into a function is called the domain.
Description:	
Obligation:	conditional Either this or **domain_of_relation** shall be used for function
Character type of values:	Upper-case Latin letters "A" through "Z", except the upper-case Latin letters "O" and "I" plus digits "0" through "9".